

Lecture 10

Midterm Exam Preparation.
Group Work: Project and Labs 5-7.

Suthep “Jogie” Madarasmi, Ph.D.

Where we are

Week	Date	Topics	Activities
10	13-15 Oct 6, 8 Oct	Midterm Review and Individual Practical Integration (Dr. Suthep)	Lab: Individual Practical Challenge & Demo (midterm integration); begin Team Sprint 1 — working vertical slice
	14-28 Oct	Midterm Exam (covers Weeks 1-7)	
11	3, 5 Nov	Cloud Deployment and Delivery to Users (Dr. Suthep)	Lab: Cloud Deployment Lab — deploy the team application to the cloud (GCP) and deliver to users; Team Sprint 2
12	10, 12 Nov	Quality Assurance, UAT, Defect Management, Maintenance, and Technical Debt (Dr. Piyanit)	Lab: QA & UAT — write UAT scripts, defect triage & fixing, maintenance tickets, and a technical-debt log; Team Sprint 3
13	17, 19 Nov	Metrics, Reviews, Retrospectives, Ethics, Responsible AI Use, and AI Usage Report (Dr. Suthep)	Lab: Metrics, code review & retrospective; Responsible-AI reflection and AI Usage Report; final project hardening.
14	24, 26 Nov	Final Delivery, Product Demonstration, and Course Integration	Lab: Final Team Demo & Submission — product demonstration and course integration
	1-11 Dec	Final Exam (covers Weeks 8-14)	
	After Finals	Project Demonstration and Presentation	

Midterm Review

Part 1. Dr. Piyanit.
25 points.

Requirements and security

Four practice activities

- 01 Stakeholder requirements
- 02 Lightweight project proposal
- 03 Role-based access control
- 04 Security landscape and vulnerabilities

Use your own notes. Explain your decisions and state any assumptions.

Four stakeholder requirements

A workshop coordinator makes the following request:

1. “The system shall let a signed-in student view all workshops that are open for registration.”
2. “The registration process should be quick.”
3. “University policy limits each student to three workshop registrations per semester.”
4. “The system shall let students cancel a registration and let coordinators see participant lists.”

Requirement analysis

Classify each statement (functional, non-functional, business objective/domain rule)

Explain the main purpose of the requirement.

Evaluate its quality (see requirement quality checklist)

Give a reason for each judgment.

Plan a check

Describe a test or measurement and the expected result.

Suggest a revision

Identify missing information and rewrite any weak requirement.

A proposal for WorkshopHub

Practice proposal

Students currently register for campus workshops through separate online forms. Coordinators manually combine the responses, which sometimes leads to duplicate registrations and delayed confirmations.

We propose WorkshopHub, a website for students and workshop coordinators. It will let students browse workshops and register, while coordinators manage participant lists. We expect fewer registration errors and less administrative work.

The first release will support free workshops only. Payment and attendance certificates will come later. The team plans to finish before the next semester.

Assess the proposal as written. Label any new details as your recommendations.

Proposal evaluation and revision

Look for these eight elements

Problem, users, solution, expected value, scope, risk, effort, and evidence of completion

Which elements are present, incomplete, or missing?
Support your judgments with phrases from the proposal.

Revise the proposal so a team can decide what to build and a reviewer can determine whether it is complete.

Two roles in WorkshopHub

Access policy

Students may view published workshops and submit registrations.

Coordinators may view published workshops, create workshops, edit workshop details, and export participant lists.

Each account has exactly one role. Roles do not inherit permissions. The system denies any operation the policy does not grant.

Your task

Design your own role–operation table showing valid permissions.
Where should the system enforce these permissions?

The security landscape

Scenario

A university assignment system allows students to upload PDF reports. It checks the file type only in the browser. A student bypasses this check and uploads a script disguised as a PDF. The server stores the file in a folder that allows script execution. When someone opens the uploaded file's URL, the server executes the script, which reads other students' reports.

Review questions

Who is the actor, and which assets need protection?

What attack surface does the student use? Where does trust change?

Which security objective is at risk: confidentiality, integrity, or availability?

Where does the uploaded content cross a trust boundary?

What controls or test would you propose?

Common vulnerabilities in SE

Topics to revisit

Broken access control and authentication failures

SQL injection and unsafe query construction

Cross-site scripting (XSS) and cross-site request forgery (CSRF)

Server-side request forgery (SSRF)

Unsafe error messages, configuration, secrets, and dependencies

For each topic

Explain how it can arise. Give a short example.

Describe a suitable defense and a check that would verify it.

Midterm Review

Part 2. Dr. Santawat.
25 points.

Project Planning & Execution: Three questions

Question	You act as	Points
1.1 WBS	Project manager	3
1.2 RACI chart	Project manager	2
1.3 Gantt chart	Project manager	5
2 Backlog grooming	Product owner	10
3 Sprint planning	Scrum master	5

Total: 25 points

What you are given

- 1 You are the project manager of DormFix, a Waterfall project that builds a web app for reporting repairs in a dormitory. Your team has four members: Aom (business analyst), Bank (UX/UI designer), Chai (developer) and Dao (developer, tester). The stakeholders outside your team are the dormitory manager (sponsor) and the maintenance head (user).

	Task (in no particular order)	Owner (role)	Days
2	A Code the request form from the approved designs	Chai Developer	4
	B Submit the requirements for approval: approved by the dormitory manager	You Project manager	1
	C Design the screens from the approved requirements	Bank UX/UI designer	3
	D Test the system: test the two coded parts together	Dao Developer, tester	2
	E Write the requirements (the first task)	Aom Business analyst	3
	F Submit both designs for approval: approved by the maintenance head	You Project manager	1
	G Design the database from the approved requirements	Chai Developer	2
	H Submit the tested system for acceptance: accepted by the dormitory manager	You Project manager	1
	I Code the technician dashboard from the approved designs	Dao Developer, tester	3

1 Team member and stakeholders

Clearly list in the task description

2 Not in order

The order is in the wording: “from the approved ...”, “the tested system”.

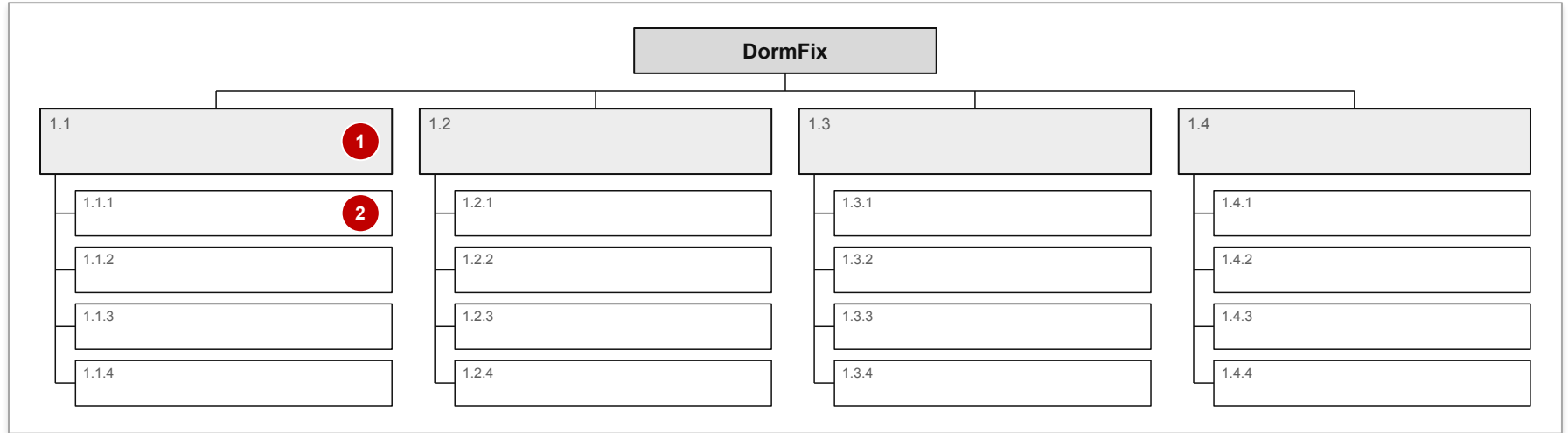
3 Owner

The name you copy into the Gantt chart.

4 Days

Working days, Monday to Friday.

WBS: what the page looks like



1 Level 1: milestone names

A short name for each milestone that you define, left to right in the order reached.

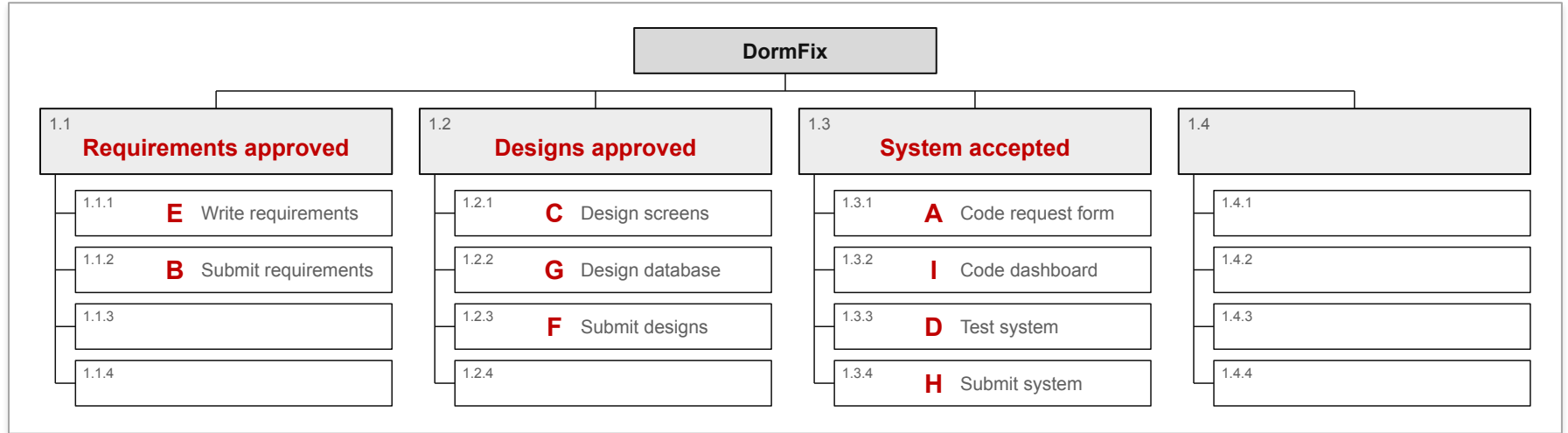
2 Level 2: task letters

The tasks needed to reach that milestone, top to bottom in the order done.

Printed rules

Use every task once. Tasks that run at the same time go in either order. Leave unused boxes empty.

WBS: a filled-in page



Three milestones

Define the name yourself based on the task

Column 1.4 stays empty

The page has more boxes than you need. Do not invent a milestone to fill them.

RACI chart

There will be a RACI chart to fill in.

Meeting notes state each person's part in each task.

Task	Task name	Person 1	Person 2	Person 3	Person 4
T1		1			
T2					
T3					
T4					

1 At least one letter in every cell

Write – when the person has no part in the task.

Gantt chart: with the facts you are given in the exam

1.3 (5 points) Plan the schedule of the tasks of 1.1 as a Gantt chart. Take the owner and the estimate (Days) of each task from the box of 1.1, and use the facts below.

Facts

- 1 • The project starts on Monday 4 January 2027; work is done Monday to Friday. A task starts on the working day after all the tasks that it depends on, and their buffers, are finished.
- 2 • You and the four team members work full-time on DormFix only. The stakeholders outside the team also have their own duties.
- 3 • Add buffers only where necessary: a task buffer of 1 day, and a project buffer of 2 days.
- 4 • A milestone is reached on the last marked day of its last task; the last milestone is reached on the last day of the project buffer.

Gantt chart: a filled-in page

No. = WBS number; Task = milestone name or task letter; Start and Due = first and last day marked. Marks: tick = working day; – = buffer day; * = milestone. Arrows = dependencies.

No.	Task	Owner	Start	Due	Week 1 (Jan)					Week 2 (Jan)					Week 3 (Jan)					Week 4 (Jan)				
					M 4	T 5	W 6	T 7	F 8	M 11	T 12	W 13	T 14	F 15	M 18	T 19	W 20	T 21	F 22	M 25	T 26	W 27	T 28	F 29
1.1	Requirements approved		8 Jan	8 Jan					*															
1.1.1	E Write requirements	Aom	4 Jan	6 Jan	✓	✓	✓																	
1.1.2	B Submit requirements	PM (you)	7 Jan	8 Jan				✓	–	2														
1.2	Designs approved		15 Jan	15 Jan											*									
1.2.1	C Design screens	Bank	11 Jan	13 Jan						✓	✓	✓												
1.2.2	G Design database	Chai	11 Jan	12 Jan						✓	✓	1												
1.2.3	F Submit designs	PM (you)	14 Jan	15 Jan										✓	–									
1.3	System accepted		29 Jan	29 Jan																		4	*	
1.3.1	A Code request form	Chai	18 Jan	21 Jan											✓	✓	✓	✓						
1.3.2	I Code dashboard	Dao	18 Jan	20 Jan											✓	✓	✓							
1.3.3	D Test system	Dao	22 Jan	25 Jan														✓	✓					
1.3.4	H Submit system	PM (you)	26 Jan	27 Jan																	✓	–		
–	Project buffer		28 Jan	29 Jan																		3	–	–

1 Overlap

C and G start together. F waits for C.

2 Task buffer

After B, F and H: outside the team.

3 Project buffer

Two days after the last task.

4 Milestone

One * and no tick.

Gantt chart: the explanation box

Explanation (1 point). In the box below, explain where you added buffers and why, and why the milestones take no working time in your chart.

I added a 1-day task buffer after B, F and H only. Each of them waits for a stakeholder outside the team to check and approve, so a delay is outside our control.

The tasks done by the team share the 2-day project buffer after the last task.

Three or four sentences are enough

Thai or English. Explain your own chart: the box must match what you drew.

Issue form: what the page looks like

Issue #12 Book a badminton court	
From the sports office (the client): Students must be able to book a badminton court for one hour, so that they do not have to queue at the counter. Courts and opening hours are already stored in the system. A student may hold only one booking per day. Example: Court 2 is free at 17:00 today; after a student books it, Court 2 at 17:00 must be shown as “Booked”.	
1 Assumptions and Details: Acceptance Criteria:	Assignees 4 Labels Milestone Estimate

1 The story

The top area has no heading.
You write who wants it, what, and why.

2 Assumptions and Details

The facts the team already knows.

3 Acceptance Criteria

In Gherkin syntax. The keywords are not printed.

4 Four fields

As they should be at the end of grooming. Write – where there is no value.

Issue form: a filled-in page

Issue #12 Book a badminton court

From the sports office (the client): Students must be able to book a badminton court for one hour, so that they do not have to queue at the counter. Courts and opening hours are already stored in the system. A student may hold only one booking per day. Example: Court 2 is free at 17:00 today; after a student books it, Court 2 at 17:00 must be shown as "Booked".

As a student

I need to book a badminton court for one hour

So that I do not have to queue at the counter

Assignees

—

Labels

enhancement

Assumptions and Details:

- Courts and opening hours are already stored in the system
- A student may hold only one booking per day

Milestone

—

Acceptance Criteria:

Given Court 2 is free at 17:00 today

When I book Court 2 at 17:00

Then Court 2 at 17:00 is shown as "Booked"

Estimate

—

Issue form: a second filled-in page

Issue #15 Cancelled bookings still block the court

Reported by students: After a booking is cancelled, the court is still shown as “Booked”, so nobody else can book it. Cancelled bookings are already saved in the database.
Example: a student cancels the booking of Court 1 at 18:00; Court 1 at 18:00 must then be shown as “Free”.

As a student

I need a court to be shown as “Free” after its booking is cancelled

So that another student can book it

Assumptions and Details:

- Cancelled bookings are already saved in the database

Acceptance Criteria:

Given Court 1 at 18:00 has been booked

When the student cancels that booking

Then Court 1 at 18:00 is shown as “Free”

Assignees

—

Labels

bug

Milestone

—

Estimate

—

Sprint backlog: what the page looks like

You are the scrum master of ShuttleNow, a campus shuttle tracking app. In sprint planning for Sprint 6, the product owner states the sprint goal: **“A student can track a shuttle and know when it arrives.”**

Facts: the team completed 16 story points last sprint and keeps the same velocity. You assign the story points (normally a team vote; its anchor story is 3 points: “As a student, I need to see the list of shuttle routes.” The developers compared each story with the anchor; “1 step larger” means the next number up on the scale.

Rank	Story in the product backlog	Compared with the anchor story	Story points	Sprint 6? (Yes / No)
1	As a student, I need to see a shuttle's live position.	2 steps larger	1	2
2	As a student, I need to pay the fare by QR code.	same size		
3	As a student, I need the arrival time at my stop.	1 step larger		
4	As a student, I need an alert before my shuttle arrives.	same size		
5	As a student, I need arrival times for all stops of a route.	1 step larger		
6	Fix: the route list is cut off on small phones.	1 step smaller		
Total story points in the Sprint 6 backlog				3

1 Story points

You assign them from the anchor story and the team's scale.

2 Yes or No

Does the story go into this sprint?
The rule is not printed.

3 Total

The story points of the stories you chose.

Then an explanation box

One point: why these stories and not the others.

Sprint backlog: a filled-in page

You are the scrum master of ShuttleNow, a campus shuttle tracking app. In sprint planning for Sprint 6, the product owner states the sprint goal: **“A student can track a shuttle and know when it arrives.”**

Facts: the team completed 16 story points last sprint and keeps the same velocity. You assign the story points (normally a team vote); its anchor story is 3 points: “As a student, I need to see the list of shuttle routes.” The developers compared each story with the anchor; “1 step larger” means the next number up on the scale.

Rank	Story in the product backlog	Compared with the anchor story	Story points	Sprint 6? (Yes / No)
1	As a student, I need to see a shuttle's live position.	2 steps larger	8	Yes
2	As a student, I need to pay the fare by QR code.	same size	3	No
3	As a student, I need the arrival time at my stop.	1 step larger	5	Yes
4	As a student, I need an alert before my shuttle arrives.	same size	3	Yes
5	As a student, I need arrival times for all stops of a route.	1 step larger	5	No
6	Fix: the route list is cut off on small phones.	1 step smaller	2	No
Total story points in the Sprint 6 backlog				16

Stories 1, 3 and 4: Yes

They support the goal. $8 + 5 + 3 = 16$, the velocity, so stop.

Story 2: No

It would fit, but paying the fare is outside the sprint goal.

Story 5: No

It supports the goal, but it would make 21, more than 16.

In the explanation box

Name the sprint goal and the velocity, and say why each “No” is out.

Midterm Review

Part 3. Dr. Suthep.
50 points.

Download Midterm Samples by Suthep.pdf

<https://docs.google.com/document/d/11DHxzZgLTwMgitFtCRvBQqeqXz0aGD0k0qevp0BY9w/edit?usp=sharing>

The screenshot shows a Google Docs interface. The title bar reads 'Midterm Samples by Suthep'. The menu bar includes File, Edit, View, Insert, Format, Tools, Gemini, Extensions, and Help. The toolbar shows search, undo, redo, print, and various text formatting options (bold, italic, underline, text color, background color). The document content is as follows:

Exam 1 Problems by Suthep Madarasmi, PhD.

1. Use the following requirements to complete various problems in this section

Requirements: KU Course Registration System

1. All users must log in with their KU account through KU Single Sign-On. User roles such as student, university admin, and instructor are provided by this system.	1. user ทุกคนต้อง log in ด้วย KU account ผ่าน KU Single Sign-On โดยที่ User roles เช่น student, university admin และ instructor จะมาจากระบบนี้
2. Department Admins can create new course information: course code, course name, description, credits, and fee.	2. Department Admins สามารถสร้างข้อมูล course ใหม่ได้: course code, course name, description, credits และ fee

UML Specification Diagrams and Git Workflow

Problems	Points
1. Given Requirements Specifications (30 points selected from below)	30
1.1 Complete a UML Sequence Diagram with swimlanes for a scenario.	10
1.2 Complete the UML Activity Diagram with partitions (swimlanes) for a workflow.	10
1.3 Prototype UI Design of List and Form Views for a Given Component	10
1.4 Draw a UML Use Case Diagram	10
1.5 Draw a UML State Diagram	10
2. Git Workflow for the ExamScore python app (20 points selected from below)	20
2.1 Predict the new repository graph after a few user transactions	10
2.2 Explain the given Repository State graph provided	10
2.3 Write Commands and Redraw the Graph	10

Total points: **50**

Exam Problems from My Lectures

Given a Minimalist Functional Requirements List:

1. Prototype UI Design of List and Form Views for a Given Component
2. Draw a UML Use Case Diagram
3. Draw UML Activity Diagram for a Given Workflow
4. Draw a UML Sequence Diagram for a Given Scenario in a Workflow
5. Draw a UML State diagram for a Given Component
- ~~6. Draw a UML Class Diagram~~

Please familiarize yourself with my example Requirements

1. POS
2. All-in Hotel Reservation
3. Software Service Ticket
4. University Grades Submission

They may be used in exam problems for you to add some features or draw certain UML diagrams. You will save time by not having to read the same requirements during in the exam.

The above 4 Requirements List will be in English only. If I use any other Requirements List, it will be provided in both Thai and English.

Git Workflow using ExamScore python app

Project Context

ExamScore is a small Python command-line application used by a university instructor. It loads student names and exam scores from a CSV file, checks that scores are between 0 and 100, calculates basic statistics, and prepares a pass-rate report. Python was chosen because its syntax is easy to read and it supports CSV processing and numerical calculations without distracting from the Git workflow.

The branch `main` represents the deployed release. The branch `sprint_1_staging` collects completed work for integration and testing. Feature branches allow developers to work separately before their changes are integrated into the sprint branch.

Files Used in the Problems

File	Purpose
<code>score_data.py</code>	Python code that loads and validates exam scores
<code>scores.csv</code>	Sample student names and score data
<code>validation_notes.txt</code>	A new file documenting validation test cases

Diagram Notation and Assumptions

- HEAD identifies the currently checked-out local branch; thus, the commit shown in the working directory.
- A name such as `origin/main` is a local remote-tracking reference. It records the last GitHub state known to this PC.
- Switching branches moves HEAD but does not move any branch pointer.
- Assume the working directory is clean, all commands succeed, and no merge conflict occurs unless the problem states otherwise.
- No command communicates with GitHub unless `fetch`, `push`, or another remote command is shown.
- Use explicit file names when staging. Do not use `git add .` in these problems.

More UML Diagram Example Problems

POS System Example

Requirements List (minimalist)

1. **Serve customers at the counter**
 - Cashiers can start a checkout, add or remove items, apply valid discounts, and see the running total.
 - A sale can be **cancelled** any time before it is finished.
2. **Accept payments**
 - Customers can pay **by cash** or **by credit card** which is processed via a **Payment Service Provider** (eg., K-Bank)
 - The system **works with the card machine** from, say, K-Bank to send the amount and receive approval
 - If a card is **declined**, the cashier can retry, switch to cash, or cancel the sale.
3. **Print documents: Receipt** for successful sale, **Refund Receipt** for refunds. Prints to store **receipt printer**.
4. Updates stock in **store's inventory service** app: **reduces** stock of the sold item, adds back stock of returned items.
5. **Handle returns and refunds.**
 - Cashiers or managers can find the original sale and **process a return**.
 - The system **refunds** the customer (cash or card, following store policy) and records the reason.
6. **Manager Functions**
 - Managers can view a **daily sales summary** for a selected date, including total cash & card amount received.
 - Managers can **add or update products and prices** to be used by POS.

POS App Actual UI (Odoo ERP shown here)

302

Conference Chair (Aluminium)
4.00 x \$ 39.40 / Units

Large Desk
1.00 x \$ 1,799.00 / Units

Desk Organizer (Leather, L)
1.00 x \$ 5.10 / Units

Total

\$ 1,961.70

Customer	Internal Note		Actions
1	2	3	Qty
4	5	6	%
7	8	9	Price
+/-	0	.	✕

Payment

Your logo

My Company (San Francisco)
Tel: +1 555-555-5556
info@yourcompany.com
http://www.example.com

Served by Mitchell Admin

302

Conference Chair (Aluminium)
4.00 x \$ 39.40 / Units

Large Desk
1.00 x \$ 1,799.00 / Units

Desk Organizer (Leather, L)
1.00 x \$ 5.10 / Units

TOTAL

\$ 1,961.70

Cash

\$ 1,000.00

Card

\$ 961.70

Generic App UI. List View.

 Point of Sale Dashboard Orders Products Reporting Configuration

Orders 

 Search...

☐ Order Ref	Session	Date	Point of Sale	Receipt Number	Custom
☐ Furniture Shop/0002	POS/00001	08/26/2025 20:44:31	Furniture Shop	Order 00003-001-0002	
☐ Furniture Shop/0001	POS/00001	08/26/2025 20:39:30	Furniture Shop	Order 00003-001-0001	
☐ ClosedDemo/0004	Furniture Shop/00001	08/24/2025 00:11:25	Furniture Shop	Order 00000-002-1002	
☐ ClosedDemo/0003	Furniture Shop/00001	08/24/2025 00:11:25	Furniture Shop	Order 00000-002-1001	
☐ ClosedDemo/0002	Furniture Shop/00000	08/24/2025 00:11:24	Furniture Shop	Order 00000-001-1002	
☐ ClosedDemo/0001	Furniture Shop/00000	08/24/2025 00:11:24	Furniture Shop	Order 00000-001-1001	

Generic App UI. Form View.

Point of Sale Dashboard Orders Products Reporting Configuration

Orders

Furniture Shop/0002 ⚙️

Pickings
1

Invoice Return Products

New Paid Posted

Order Ref Furniture Shop/0002

Date 08/26/2025 20:44:31

Session POS/00001

User ? Mitchell Admin

Customer

Fiscal Position

Products

Payments

Extra Info

General Notes

Products

Payments

Extra Info

General Notes

Date	Payment Method	Amount
08/26/2025 20:44:24	Card	\$ 961.70
08/26/2025 20:44:18	Cash	\$ 1,000.00

Add a line

Product	Lot/serial Nu...	Quantity	Unit Price	Disc.%	Taxes	Tax Excl.	Tax Incl.	🔍
[E-COM13] Conference Chair (Aluminium)		4.00	\$ 39.40	0.00		\$ 157.60	\$ 157.60	
[E-COM09] Large Desk		1.00	\$ 1,799.00	0.00		\$ 1,799.00	\$ 1,799.00	
[FURN_0001] Desk Organizer		1.00	\$ 5.10	0.00		\$ 5.10	\$ 5.10	

Taxes: \$ 0.00

Total: \$ 1,961.70

Total Paid (with rounding): \$ 1,961.70

UI Menu

The screenshot displays a software interface with a top navigation bar and a main content area. The navigation bar includes the following items: a barcode icon, 'Point of Sale', 'Dashboard', 'Orders' (highlighted with a blue border), 'Products', 'Reporting', and 'Configuration'. Below the navigation bar, the 'Orders' section is visible, featuring a gear icon. A dropdown menu is open under the 'Orders' tab, listing the following options: 'Orders', 'Sessions', 'Payments', 'Preparation Display', and 'Customers'. The main content area below the dropdown shows a table with two rows of data. The first row has a checkbox, 'Order Ref', and a 'Date' header. The second row has a checkbox, 'Furniture Shop/0002', and the date '08/26'.

☐	Order Ref	Date
☐	Furniture Shop/0002	08/26

Product List of Value (LoV) via List View

Search: Product



Product

Desk



Search...



1-12 / 12



Internal Refere...	Name	Variant Values	Sales Price	Cost	On Hand	Forecasted	
☆ E-COM06	Corner Desk Right Sit		147.00	600.00	0.00	0.00	
☆ E-COM09	Large Desk		1,799.00	1,299.00	-1.00	11.00	
☆ FURN_0001	Desk Organizer		5.10	0.00	-1.00	-1.00	

New

Close

POS Sales Form View Generic Design (Minimalist)

POS Sales Form View Generic Design

User: Ying Sanrungroengkijthaveepop Role: Cashier

Sales Return Products Set Prices Logout Reports Authorization (App Controls)

[Close][New][Save][Clear][Print] (Form Controls)

Invoice No.: INV25-001 *Invoice Date: _07/01/2025_ [Date]

Customer Code: _Walk-in_ [LoV] Customer Name: No Name

#	*Product Code	Product Name		*Quantity		Unit Price	Units	Extended Price	Edit		
1	DESK-11	Work Desk Wooden		1.00		6,000.00	PCS	6,000.00	Edit	✕	▲
2	CHAIN-1	Link Chain 5mm		10.50		100.00	METER	1,050.00	Edit	✕	□
3	CHAIR-2	Office Chair		2.00		2,000.00	PCS	4,000.00	Edit	✕	
4	[LoV]		[-1]	1.00	[+1]						▼
Total:								11,050.00			

POS Sales List View Generic Design (Minimalist)

POS Sales List View

User: Ying Sanrungroengkijthaveepop Role: Cashier

Sales Return Products Set Prices Logout Reports Authorization (App Controls)

[Close][New]

Search / Group /Filter: _____ [OK]

Next Action: ☒ Open Form (Primary Key) ☐ Select Record (Foreign Key)

Click Row to select a record: (Will open form or select the record, depending on “Next Action”)

Invoice No.	Date	Customer Name	Total Amount
INV24-721	27/12/2024	No Name	5,000.00
INV24-722	28/12/2024	No Name	1,000.00
INV24-723	28/12/2024	No Name	2,000.00
INV25-001	07/01/2025	Chaloem Sukh Co., Ltd	10,000.00

Product List View Generic Design (Minimalist)

Product List View

User: Ying Sanrungroengkijthaveepop Role: Cashier

Sales Return Products Set Prices Logout Reports Authorization (App Controls)

[Close][New]

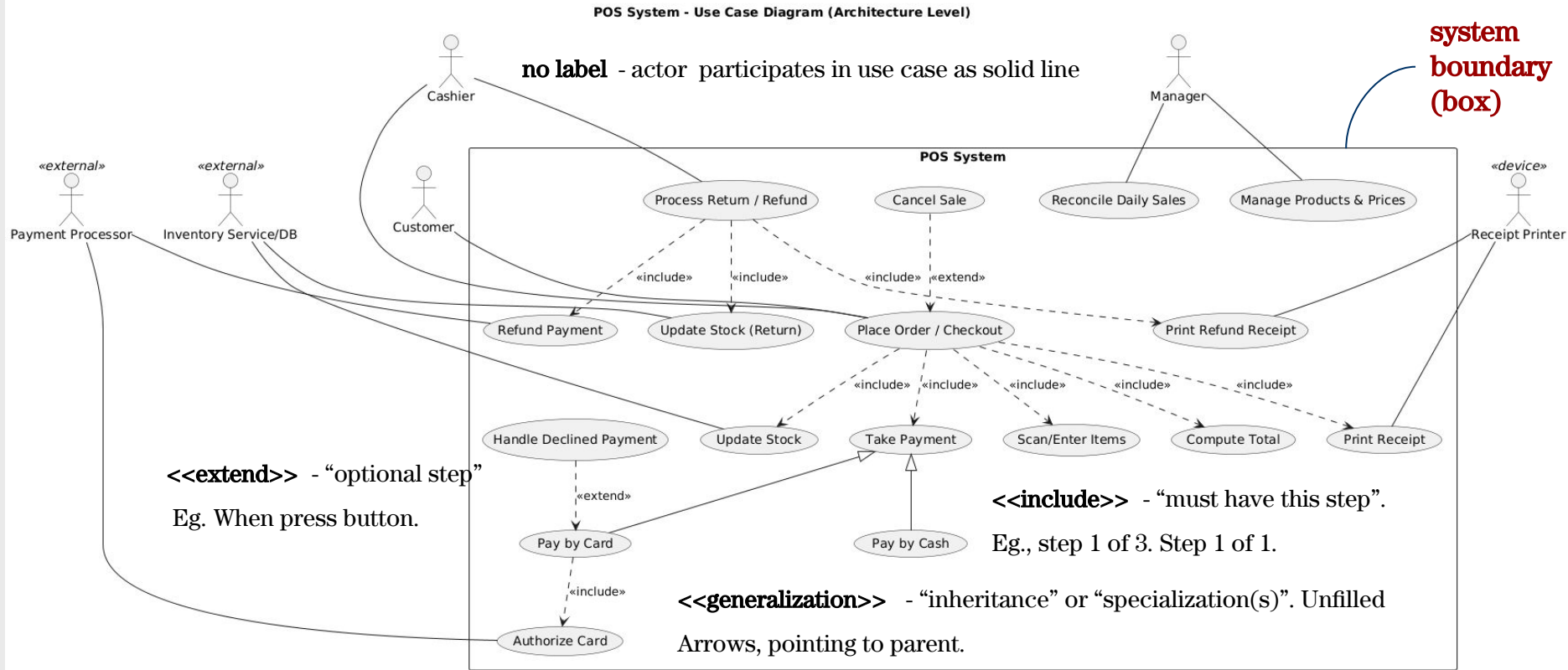
Search / Group /Filter: _____ [OK]

Mode: ☐ Action Menu (Primary Key) ☒ Select Record (Foreign Key)

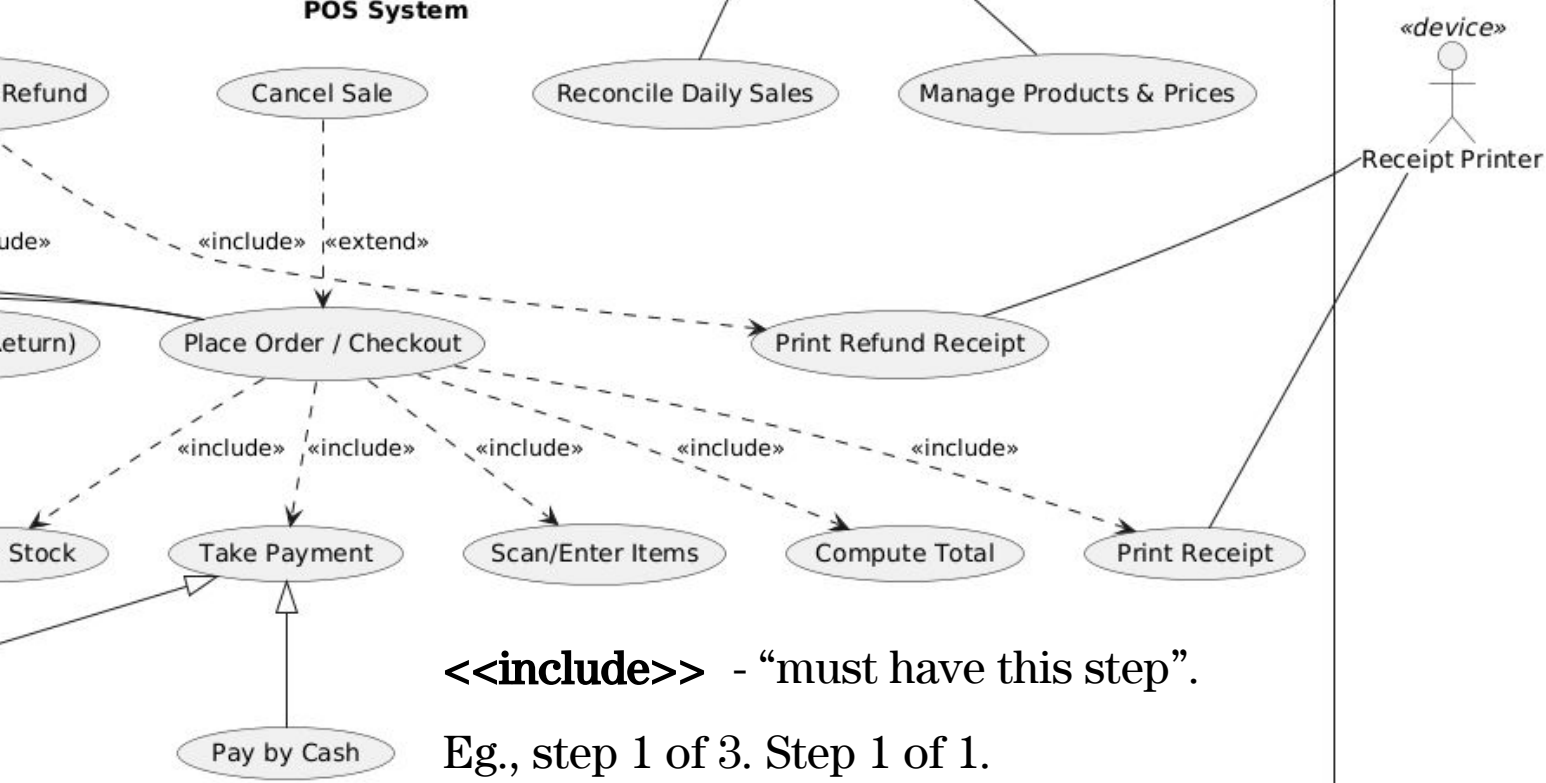
Click Row to select a record: (Will open form or select the record, depending on “Next Action”)

Product Code	Name	Unit Price	Units
CHAIN-1	Link Chain 5mm	100.00	METER
CHAIR-1	Dining Table Chair	1,500.00	PCS
CHAIR-2	Office Chair	2,000.00	PCS
COUCH-1	Leather sofa couch	20,000.00	PCS
DESK-11	Work Desk Wooden	6,000.00	PCS

POS System Use Case Diagram



Sample Exam Problem: Draw a UML Use Case diagram at the architectural level. For the main workflows “Checkout” and “Refund”, go 1-2 levels deeper to cover the entire workflow.

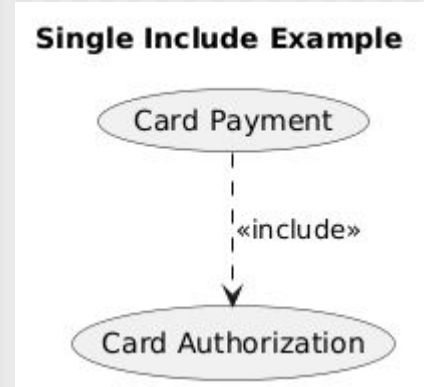
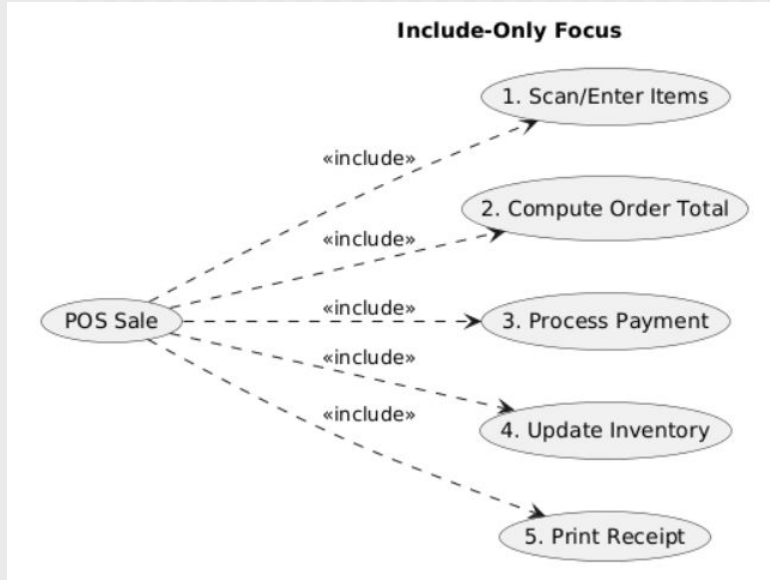


<<include>> - “must have this step”.

Eg., step 1 of 3. Step 1 of 1.

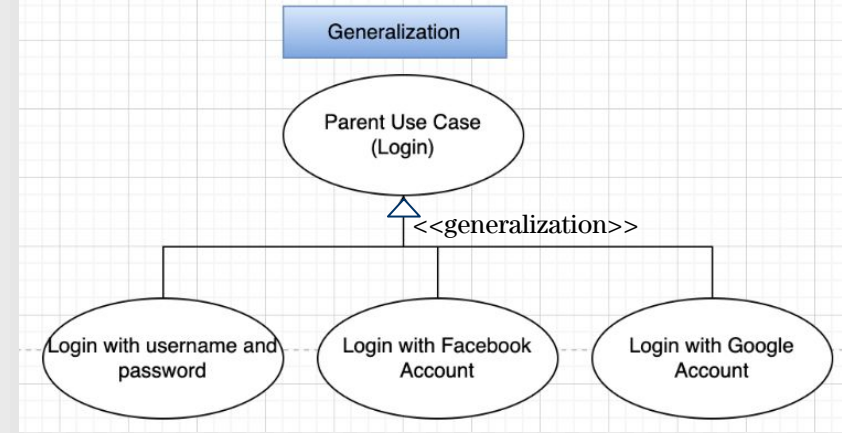
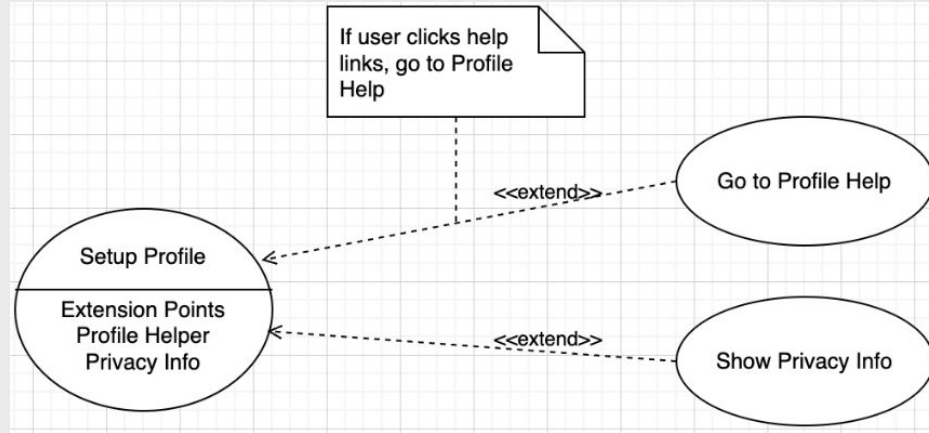
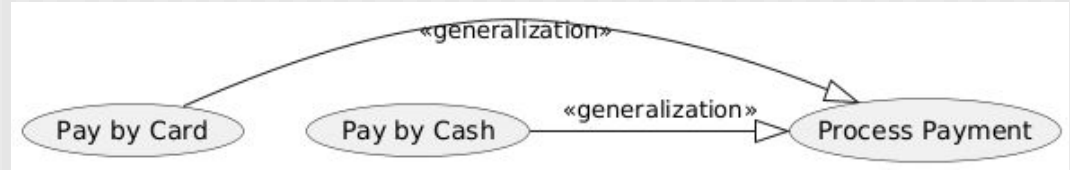
<<realization>> - “inheritance” or “specialization(s)”. Unfilled triangle pointing to Parent.

Link labels in Use Case Diagrams



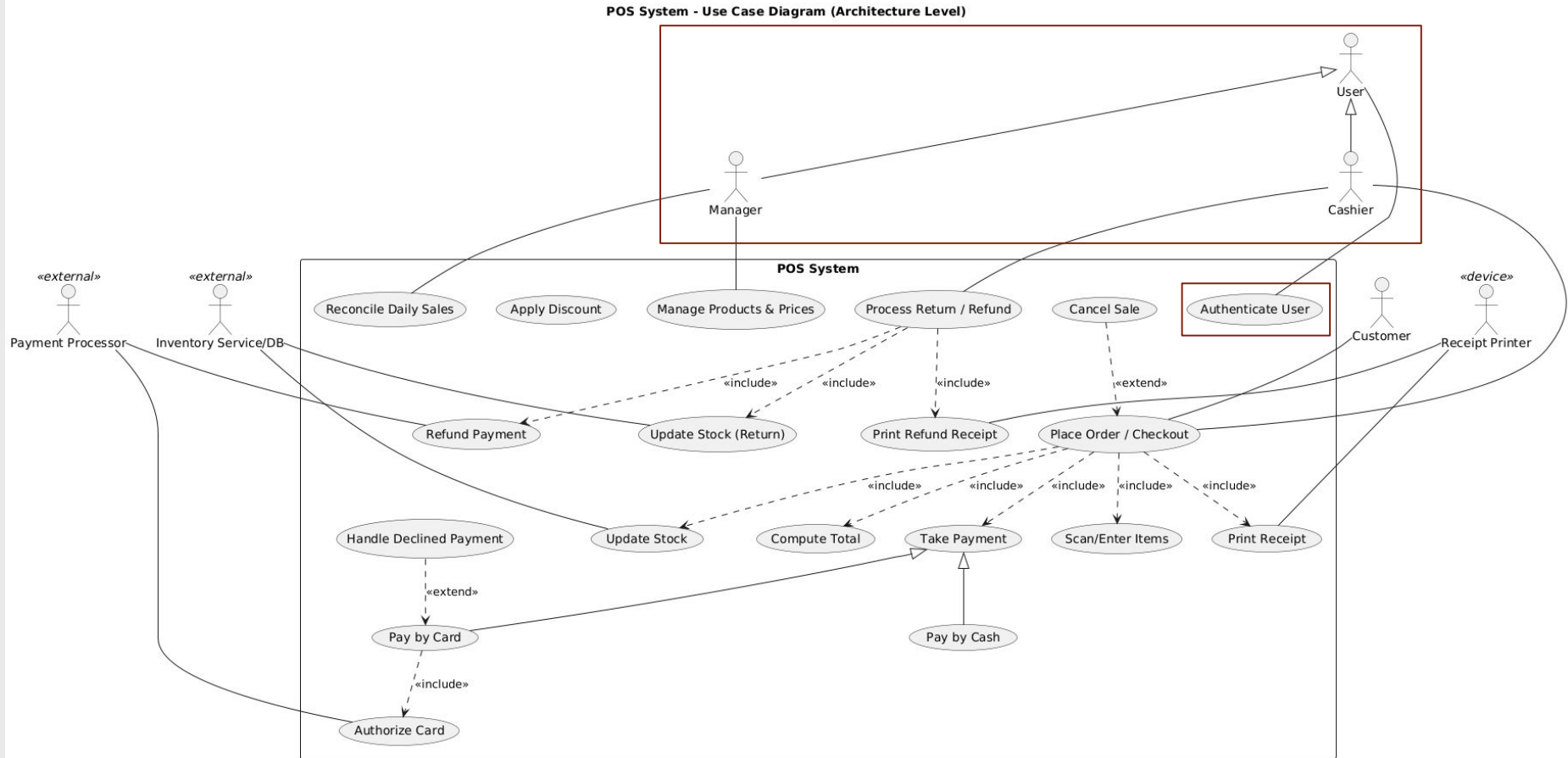
1. **no label** - actor participates in use case as solid line, no arrows.
2. **<<include>>** - Subprocesses. “must have this step n as a part of m steps”. May be 1 of 1.
3. **<<extend>>** - Optional. “does this if some condition met eg., press button”
 - “option n of m”. Arrow points from extended use case to the main use case.
4. **<<generalization>>** - “inheritance” or “specialization(s)”. Users: Manager, Cashier.

Link labels in Use Case Diagrams



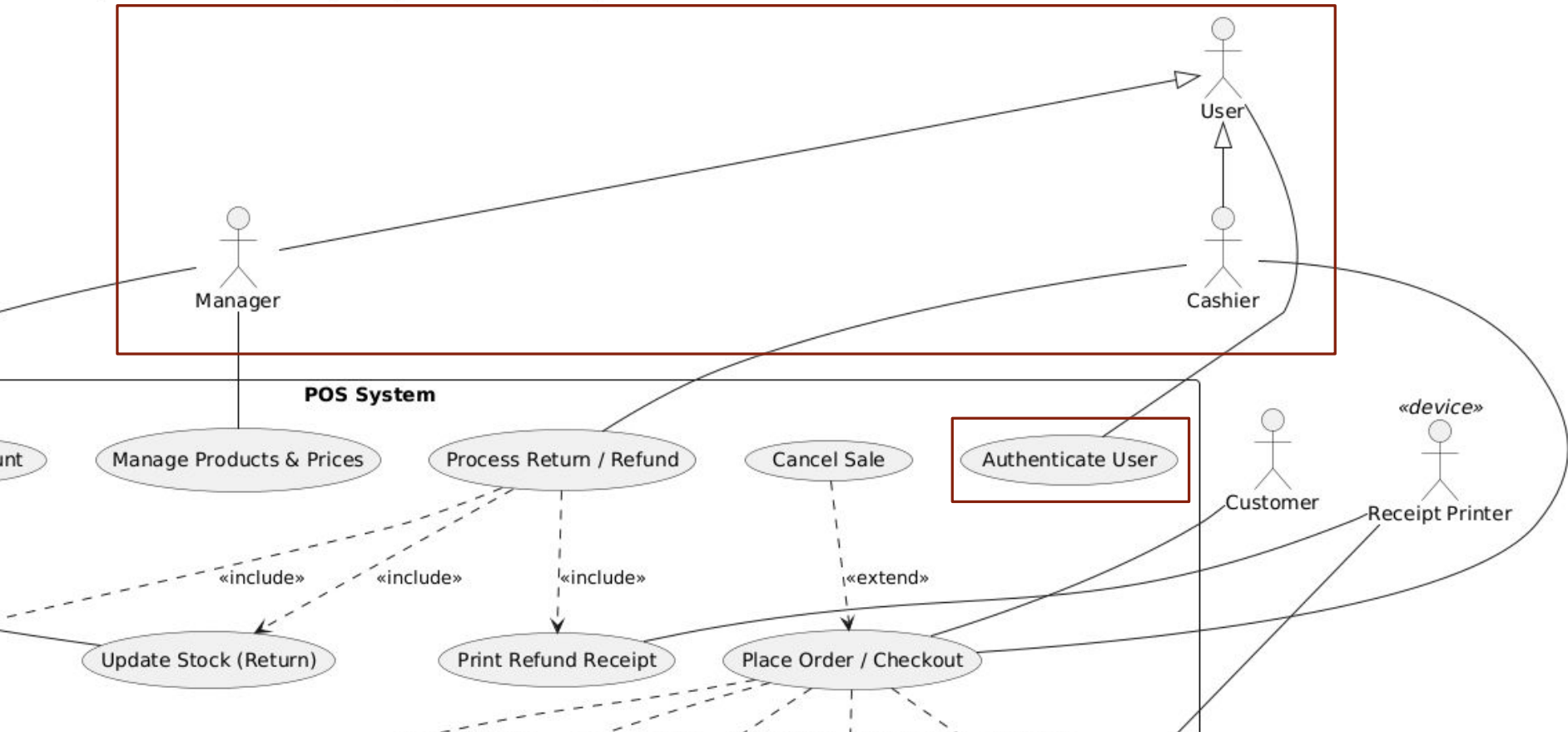
1. **no label** - actor participates in use case as solid line, no arrows.
2. **<<include>>** - “must have this step n as a part of m steps”. May be 1 of 1.
3. **<<extend>>** - Optional. “does this if some condition met eg., press button”
- “option n of m”. Arrow points from extended use case to the main use case.
4. **<<generalization>>** - “inheritance” or “specialization(s)”. Users: Manager, Cashier.

Showing Manager/Cashier as Users. Users Must Authenticate.



Showing Manager/Cashier as Users. Users Must Authenticate.

POS System - Use Case Diagram (Architecture Level)



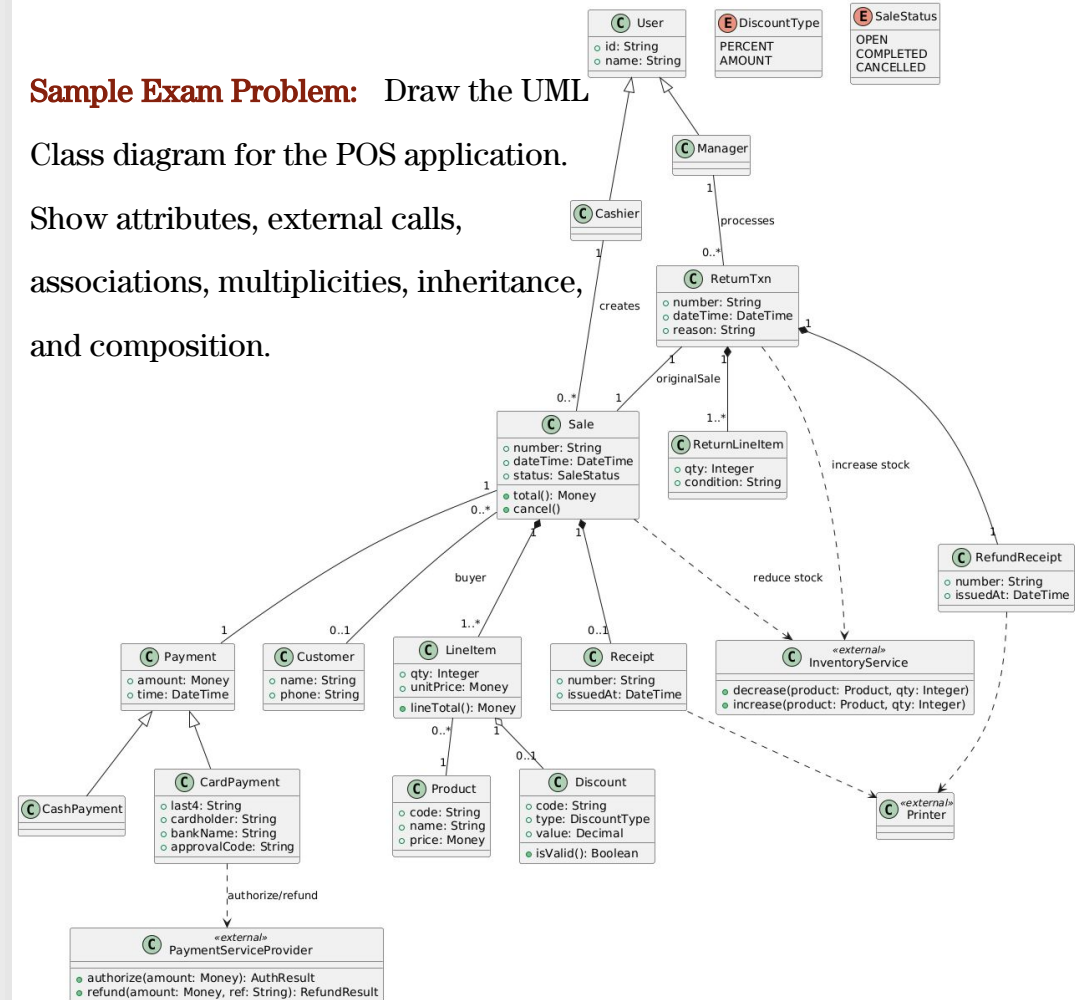
POS Class Diagram

- Attributes** = class fields + their types.
- Associations + multiplicities** = relations with 1..*, 0..1, etc. Use dashed arrows for external links.
- Composition** = strong whole-to-part links. “owns-a, born/die together”.
 - Sale ◆-- LineItem. Sale ◆-- Receipt
 - Return ◆-- ReturnLineItem, Return ◆-- RefundReceipt
- Aggregation** = white filled diamond ◇-. Weak whole-to-part links. No need to use it. **Aggregation.**
- Important inheritance Generalization** arrows (unfilled head) point to the **parent**.
 - Cashier, Manager → User.
 - CashPayment, CardPayment → Payment.

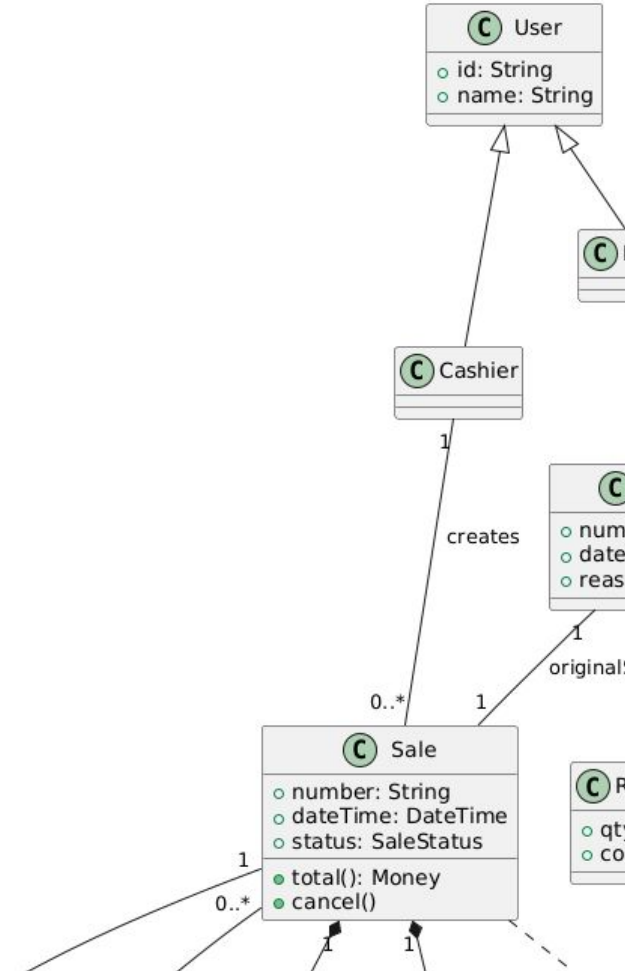
Sample Exam Problem: Draw the UML

Class diagram for the POS application.

Show attributes, external calls, associations, multiplicities, inheritance, and composition.



1. **Attributes** = class fields + their types.
2. **Associations + multiplicities** = relations with 1..*, 0..1, etc. Use dashed arrows for external links.
3. **Composition** = strong whole-to-part links. “owns-a, born/die together”.
 - a. Sale ◆-- LineItem. Sale ◆-- Receipt
 - b. Return ◆-- ReturnLineItem, Return ◆-- RefundReceipt
4. **Aggregation** = white filled diamond ◇-- Weak whole-to-part links. No need to use it. ~~Aggregation.~~
5. **Important inheritance Generalization** arrows (unfilled head) point to the **parent**.
 - 1) Cashier, Manager User.
 - 2) CashPayment, CardPayment Payment.



Workflow vs. Scenario using Activity vs. Sequence Diagrams

Workflow (Activity Diagram) = the **whole business process** with branches.

Scenario (Sequence Diagram) = **one concrete path** through that workflow. Example:

UI → Controller → Payment Service → Bank API

1. **Basic Flow / Main Flow** → the standard, expected sequence of steps when everything goes right.
2. **Alternative Flow** → valid variations via another “if/else” path (e.g., choosing Bank Transfer instead of Credit Card).
3. **Exception Flow** → error or failure flows (e.g., payment declined, invalid password).

POS **Order** Workflow. Workflow vs. Scenarios.

Activity Diagram: Workflow (with branches for options)

1. **Start** → **Scan/Add items** → Show cart total
2. **Apply discounts?** → [Yes] validate & apply → [No] continue
3. **Choose payment method** → Cash | Card | E-wallet
4. **Handle payment**
 - Cash → accept cash → give change
 - Card/E-wallet → call payment service → auth result
5. **Payment success?**
 - [Yes]: print/issue receipt → update inventory → End
 - [No]: **Offer change to cash** → if cancel, void order → End

Example Core POS Scenarios

1. Place Order / Checkout
2. Pay by Cash
3. Pay by Card (with Authorization)
4. Cancel a Sale (Midway)
5. Process Return / Refund
6. Print Receipt
7. Update Stock for Sales & Returns

Sample Scenario Paths in the POS **Order** Workflow.

1. Success Scenario: Pay by Cash (Happy Path)

- **Sequence (participants):** Customer → Cashier/POS UI → POS App → Inventory → Receipt Printer
- **Messages:** addItem* → computeTotal → recordPayment(cash, amount) → updateStock → printReceipt

2. Success Scenario: Pay by Card with Online Auth (Alt path)

- **Participants:** Cashier/POS UI → POS App → Payment Service → Inventory → Receipt Printer
- **Messages:** authorize(amount, cardToken) → authOK → capture → updateStock → printReceipt

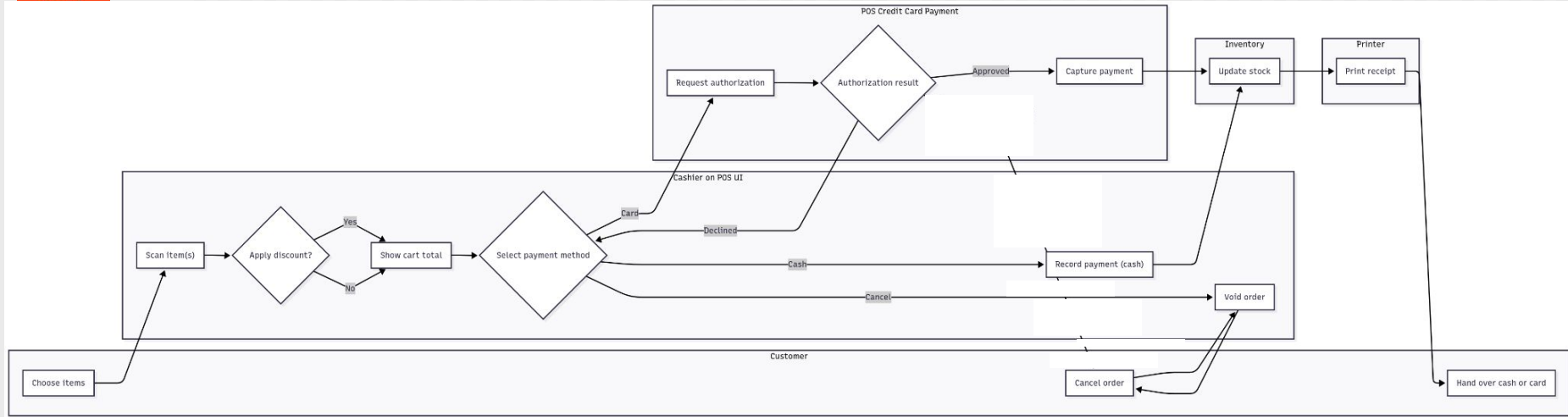
3. Card Declined Scenario and Switch to Cash (Alt path)

- **Messages:** authorize → authFailed → alternatePayment? yes by cash → recordPayment(cash) → updateStock → printReceipt

4. Out-of-Stock Detected at Checkout and Item Changed (Alt path)

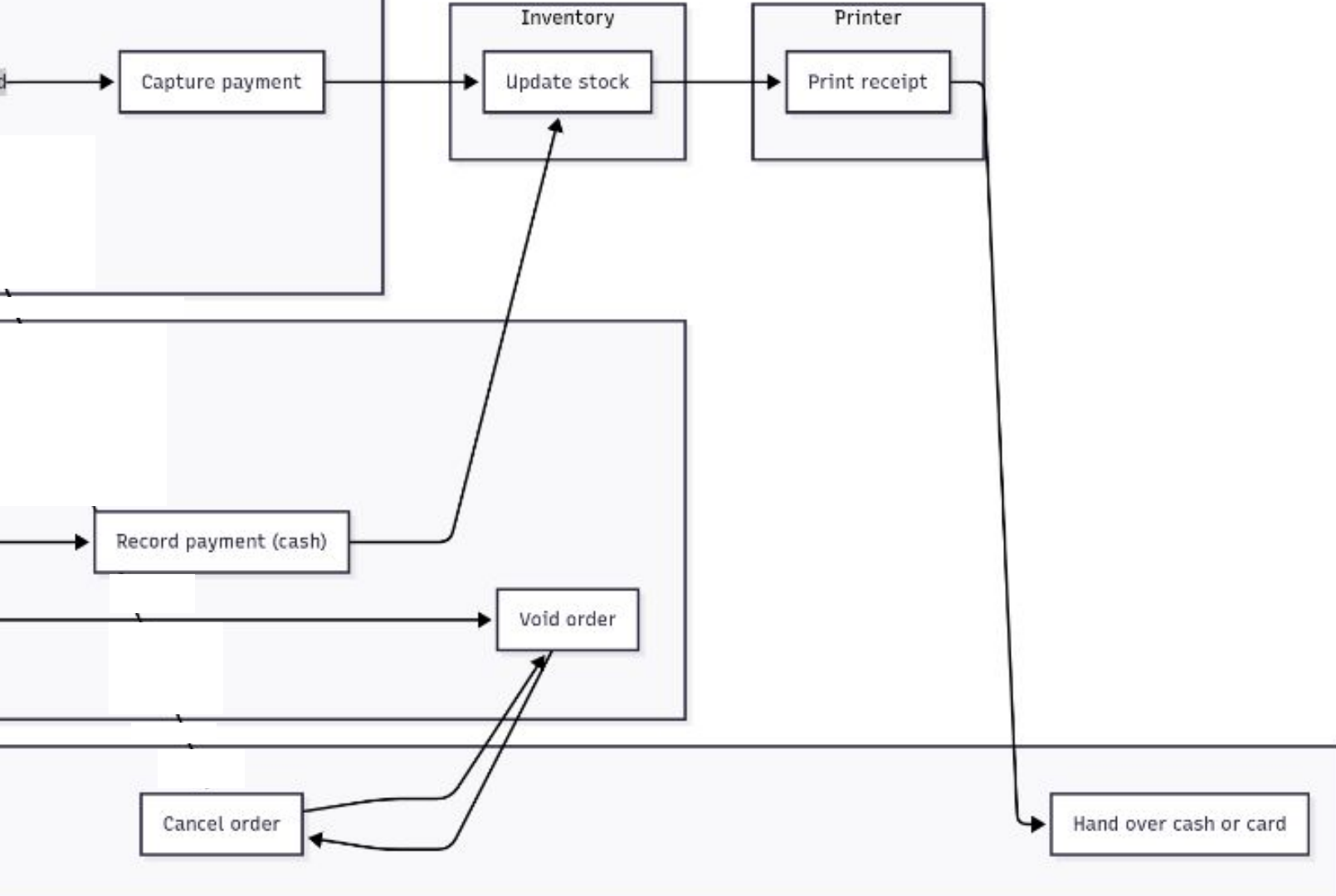
- **Messages:** addItem → checkAvailability → outOfStock → Remove → recomputeTotal → go to payment

Activity Diagram for POS Order Workflow

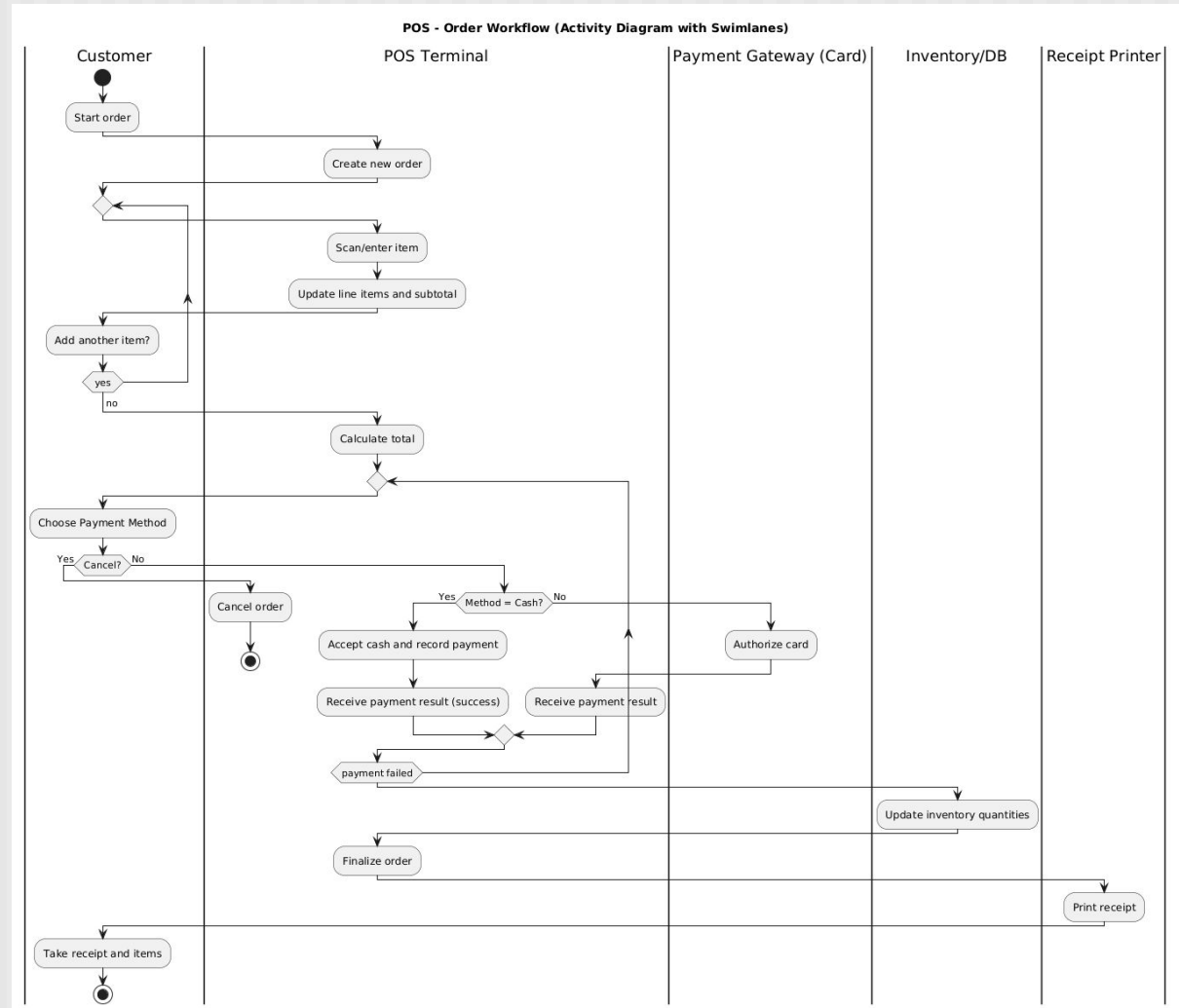


“Each swimlane is a **partition** representing a responsible Actor (user, role, external component). The actions inside the lane are owned by that partition.”

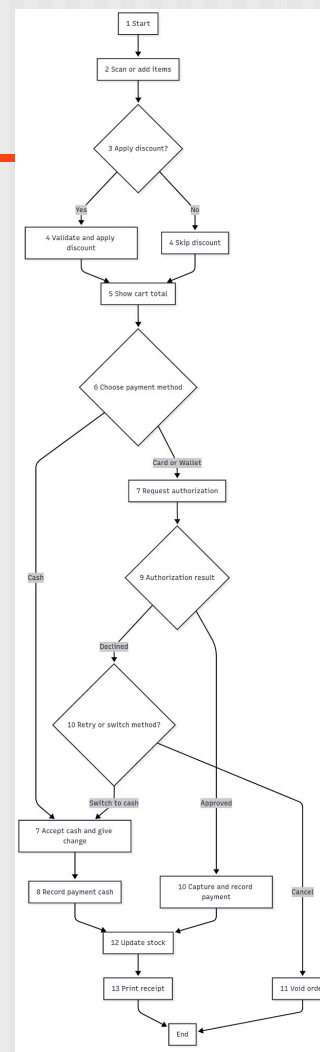
Sample Exam Problem: Draw the UML Activity diagram with partitions (swimlanes) for the POS Order workflow from select items to payment. Note that 1) customers' card payment may be declined 2) customer may decide to cancel before payment.

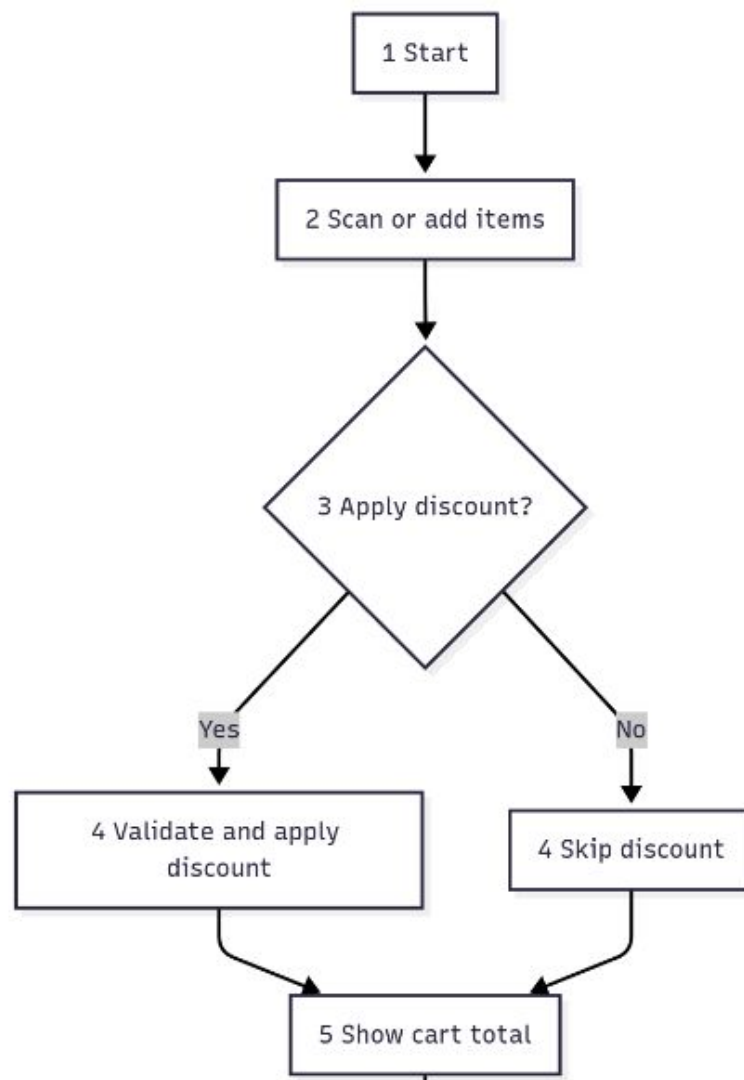


Activity Diagram for POS Order Workflow: Vertical Swimlanes.

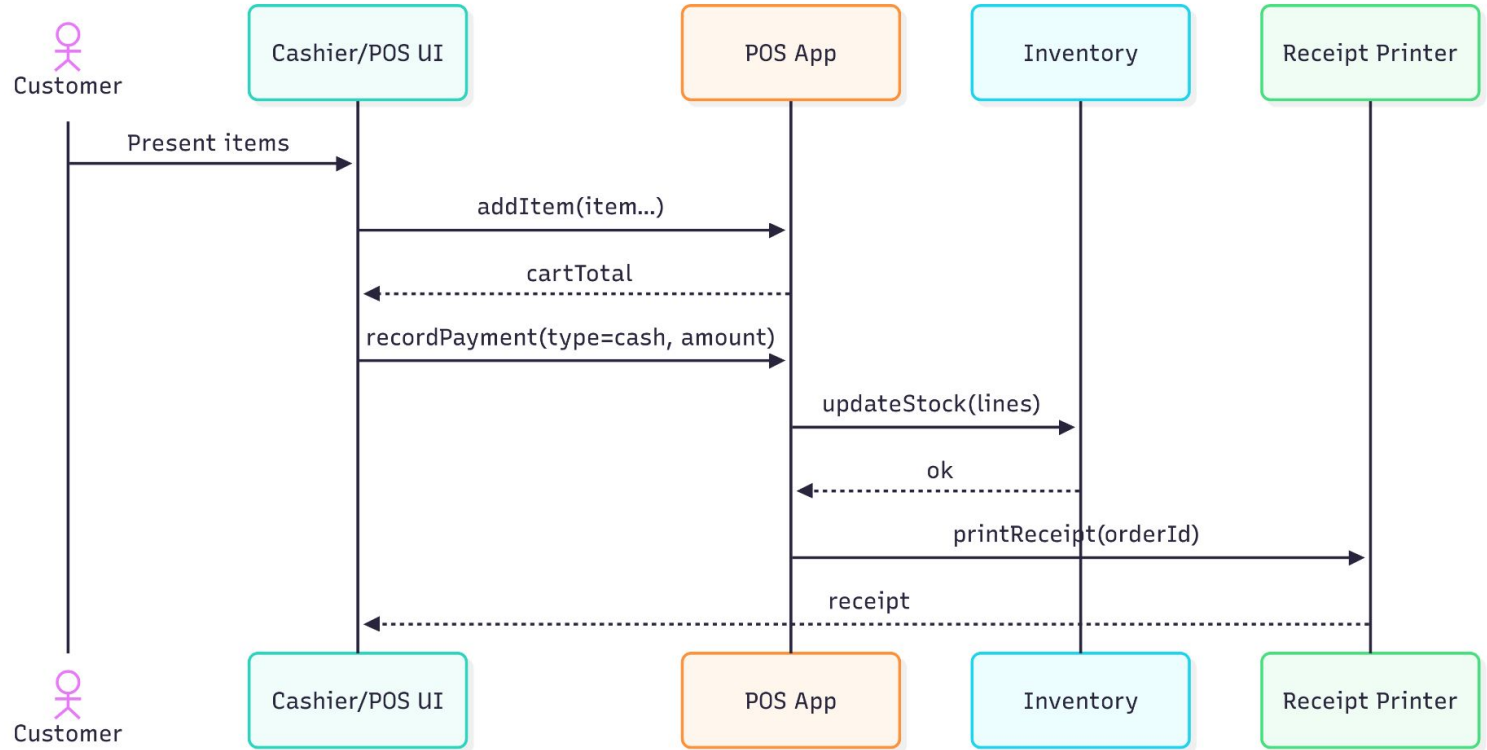


Activity Diagram (No Swimlane)





Sequence Diagram: Pay by Cash Scenario

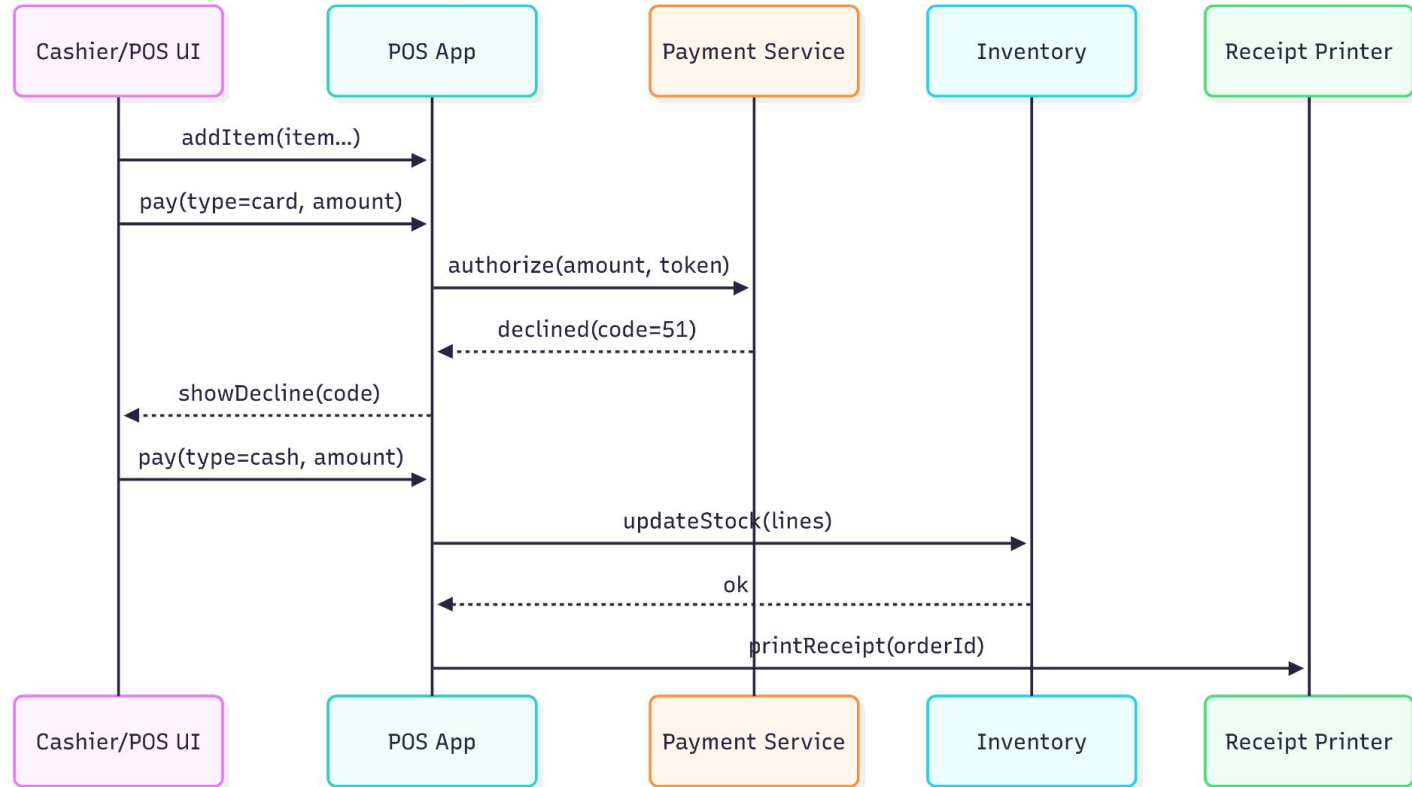


Sample Exam Problem: Draw the UML Sequence Diagram for scenario: Customer successfully buys items, paying by cash.

Sequence Diagram: Credit Card Declined, Switch to Cash Scenario

Sample Exam

Problem: Draw the UML Sequence Diagram for scenario: Customer buys items, paying by credit card, card declined, switches to pay by cash.



POS Returns and Refunds Workflow . Workflow vs. Scenarios.

Activity outline for Workflow.

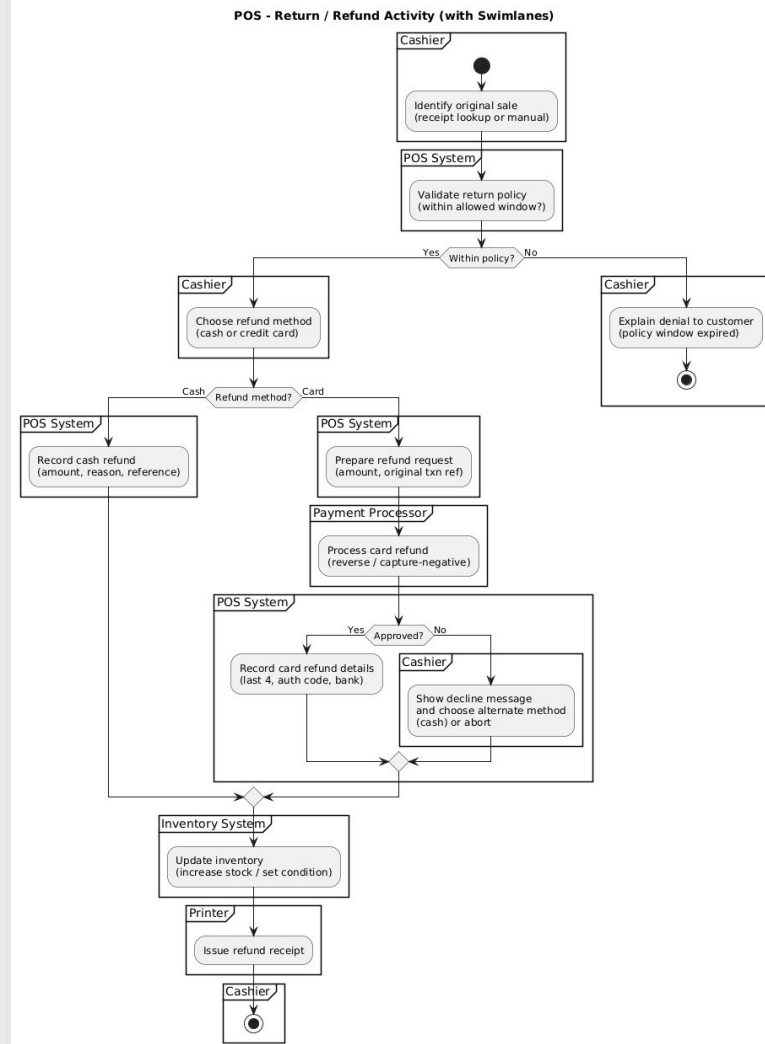
1. **Start** → **Identify original sale** (receipt lookup/manual)
2. **Validate return policy** (window, condition)
3. **Choose refund method** (original tender, store credit)
4. **Execute refund** (reverse/capture-negative)
5. **Update inventory** (return-to-stock or damaged) → Issue refund receipt → **End**

Scenarios

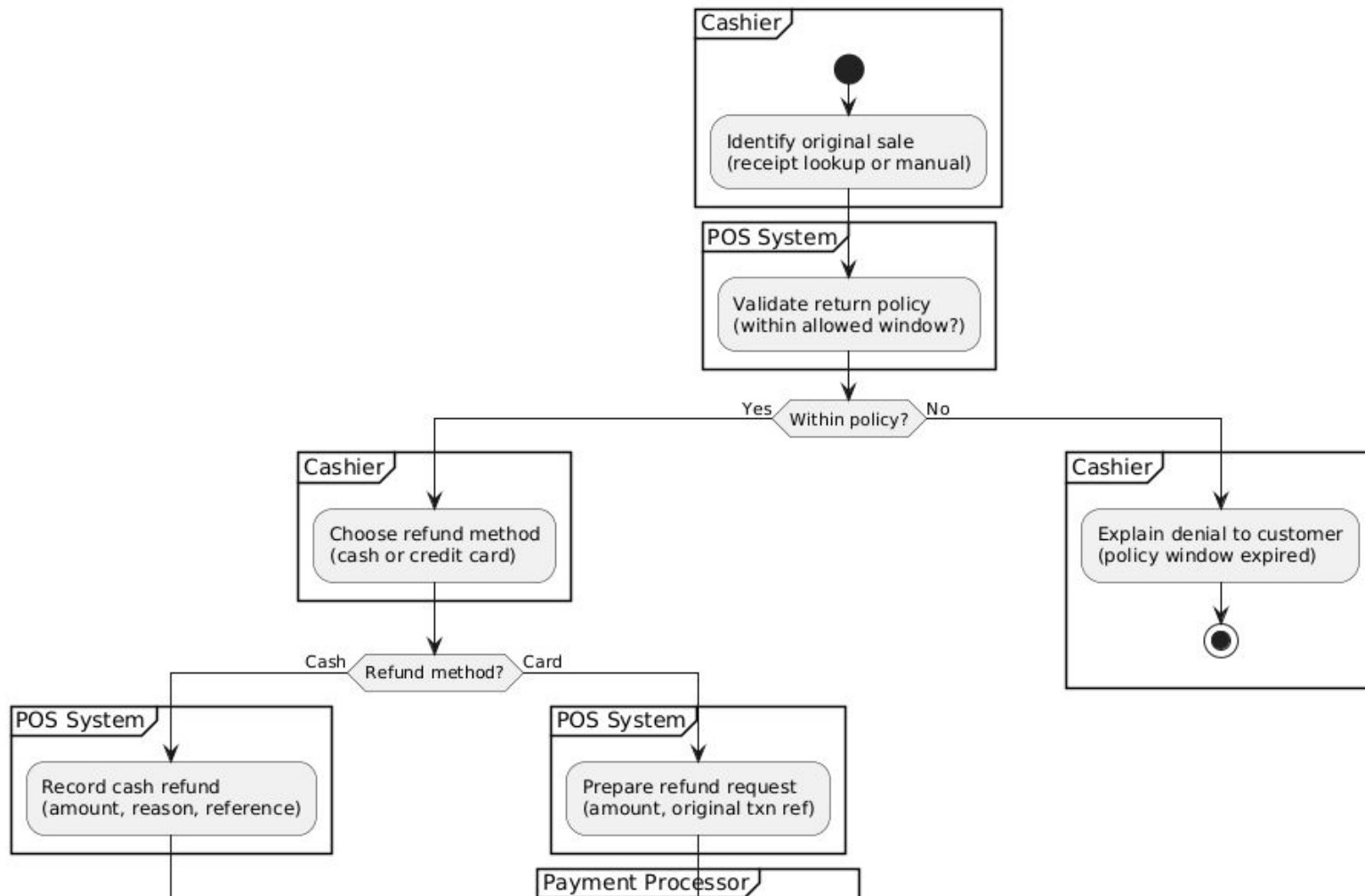
1. **Refund to Original Card (Happy Path). Sequence:** refundRequest → validatePolicy → refund(cardTxnId, amount) → refundOK → restock → printRefundReceipt
2. **Refund Denied: Too many days past. Sequence:** validatePolicy → denyReturn(reason) → offerStoreCredit? → End
3. **Partial Refund On Only One Line Item. Sequence:** selectLine → compute Pro-rated Tax → refund Partial → restockPartial

POS Return/Refund Activity Diagram with Partitions (swimlanes)

Sample Exam Problem: Draw the UML Activity Diagram for workflow: Customer returns items. Note return may be denied if against store return policy. Also, card payment may have problems.



POS - Return / Refund Activity (with Swimlanes)



Hotel Reservation Example

Functional Requirements (FR): All-in Hotel Reservation System

All-Inn is a simple online booking system where guests can reserve rooms at many member hotels through <http://www.all-inn.com>. All-Inn takes a 10% commission for each booking from the member hotel. The system supports three Room Types for every member hotel: Regular Room, Superior Room, Suite Room. The system uses Microsoft Outlook/SMTP as the external Email Service to send emails to users.

1. All Users (Guest, All-in Admin, Hotel Admin)

- Log in.
- Change own password.
- Request reset for forgotten password. Automated email link will be sent to user.

2. Guest User (Guests are registered All-Inn members)

- Search for rooms by entering check-in date, check-out date, city, and country.
- View a list of hotels with room types and total price (sum of nightly rates). Options: View Detail or Reserve.
- View hotel details with pictures, room type details with pictures, nightly prices, total price, and hotel location shown via integration with Google Maps.
- Make a reservation: see summary (hotel, room type, nights, prices), then edit name, email, and contact info.
- Pay by credit card via K-Bank Payment Gateway.
- View on-screen booking confirmation with confirmation number with booking info.
- System reduces available rooms and sends confirmation email to the hotel and to the guest.
- Other functions: Cancel My Reservation, View My Reservations, Change My Profile.

Condensed FR: All-in Hotel Reservation System

3. All-Inn Admin User

1. Create new user accounts: Guest, Hotel Admin, or All-Inn Admin.
2. Reset any user's password. Email link will be sent to user.
3. Add new hotels: hotel code, name, location, city, country, pictures, description, email.
4. Add room types for hotels: hotel, room type, description, pictures.
5. View **Room Summary Report** for a selected hotel and date range (hotel, date, room type, rooms supplied, rooms reserved, rooms available, unit price, total reserved value, total commission value).

4. Hotel Admin User

1. One admin user can manage only one hotel.
2. Set up daily room supply and price (hotel, date, room type, rooms supplied, unit price).
3. View reservations by All-in starting on a given check-in date (reservation details, guest info, total price).
4. View **Room Summary Report** for their own hotel over a selected date range.

Hotel UI.

Show a Hotel List View after selecting Date Range and City

Hotels List View

Guest: Sing Songkram

[Close]

Country: _Thailand_ [LoV]

Province: _Chonburi_ [LoV] City: _Pattaya_ [LoV]

Check-In Date: __07/06/2025_ [Date] Check-Out Date: __10/06/2025__ [Date]

Number Nights: 3 Room Type: ☒Regular ☒Special ☐Suite

[FILTER][CLEAR]

List of Hotels with Availability for Above Selection: (select row to view hotel/room information)

Hotel Name	Address	Lowest Priced Room / Night
Holiday Villa	121, Pattaya North, Thailand	2,000.00
Cheapo Hotel	221, Pattaya South, Thailand	700.00
Quality Beds	71, Pattaya South, Thailand	1,500.00
Haunted Inn	21/49, Pattaya, Thailand	350.00

Sample Exam

Problem: Show a simple Hotels List View UI. Guest selected country, province, city plus the check-in, check-out dates. System then lists the qualified hotels with starting prices for the nights selected. Guest can select hotel to view room price details.

Hotel UI. Room booking view after Dates / Hotel is selected

Reservation Form View Generic Design

Guest: Sing Songkram

[\[Back to Select Hotel/Dates\]](#) [\[Make Payment\]](#)

Booking No.:

Booking Date:

Check-in Date:

Check-out Date:

Number Nights:

Guest User:

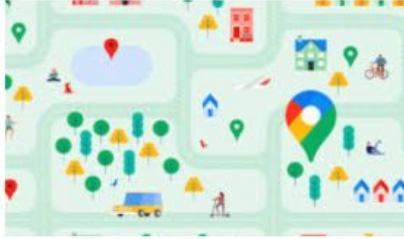
Hotel:

City:

Hotel Image:



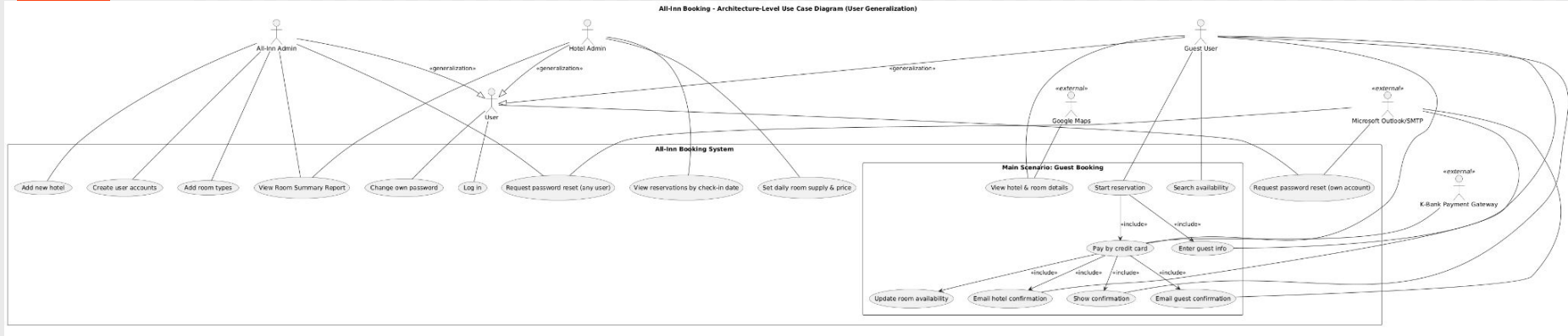
Hotel Map View:



Room Type	Total Price	Average Price / Night	Number Rooms Available	*Number Rooms To Book	Your Price	
Regular	6,000.00	2,000.00	3	2	12,000.00	View Image
Superior	9,000.00	3,000.00	8	1	9,000.00	View Image
Suite	12,000.00	4,000.00	1			View Image
Your Total Price:					21,000.00	

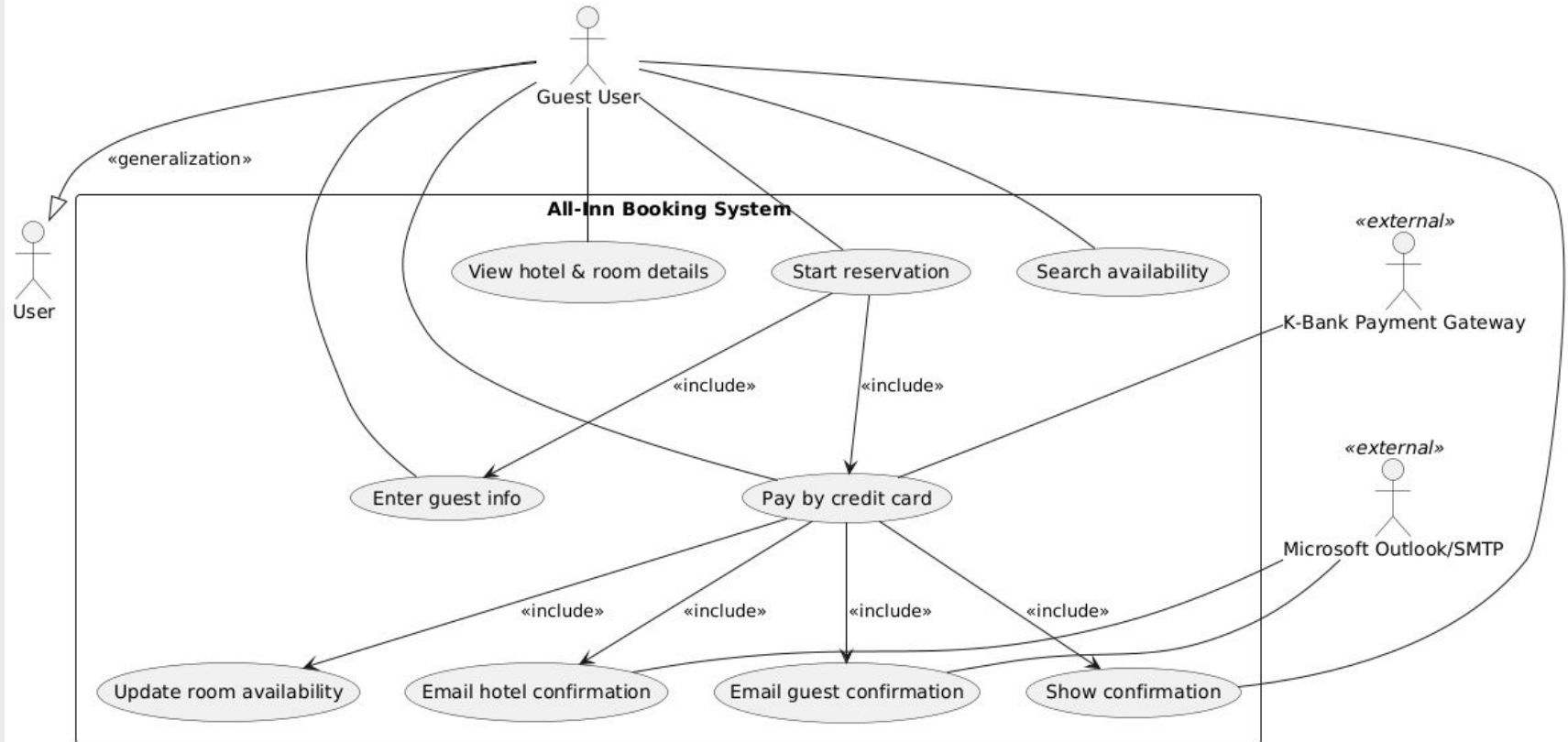
Sample Exam Problem: Show the Booking Form UI where hotel information including image and Google Map panel is presented along with available rooms. Assume the hotel and date ranges have already been selected from a previous list view. Each available room type and price for the nights of stay (e.g., 2 nights) will appear. In one booking, the guest can choose many rooms for each room type.

Hotel Reservation Use Case Diagram

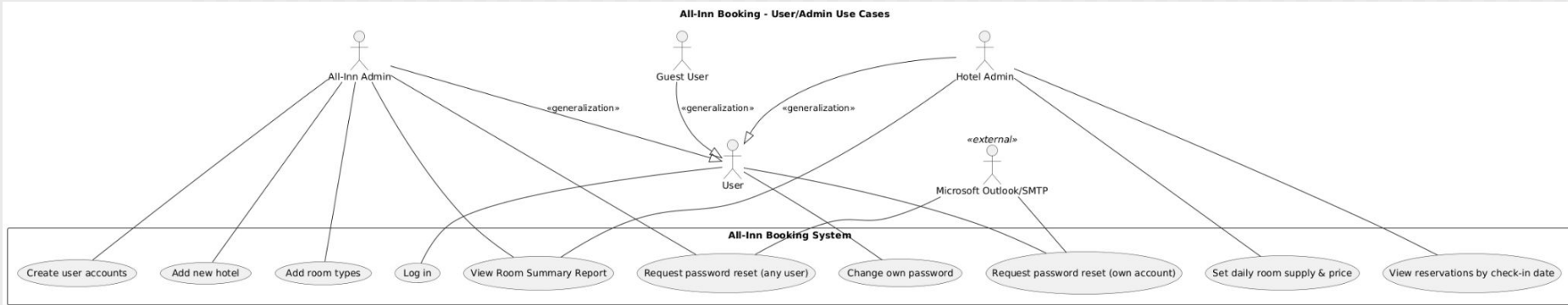


Hotel Reservation Use Case: Zoom-In Right

All-Inn Booking - Guest User Use Cases

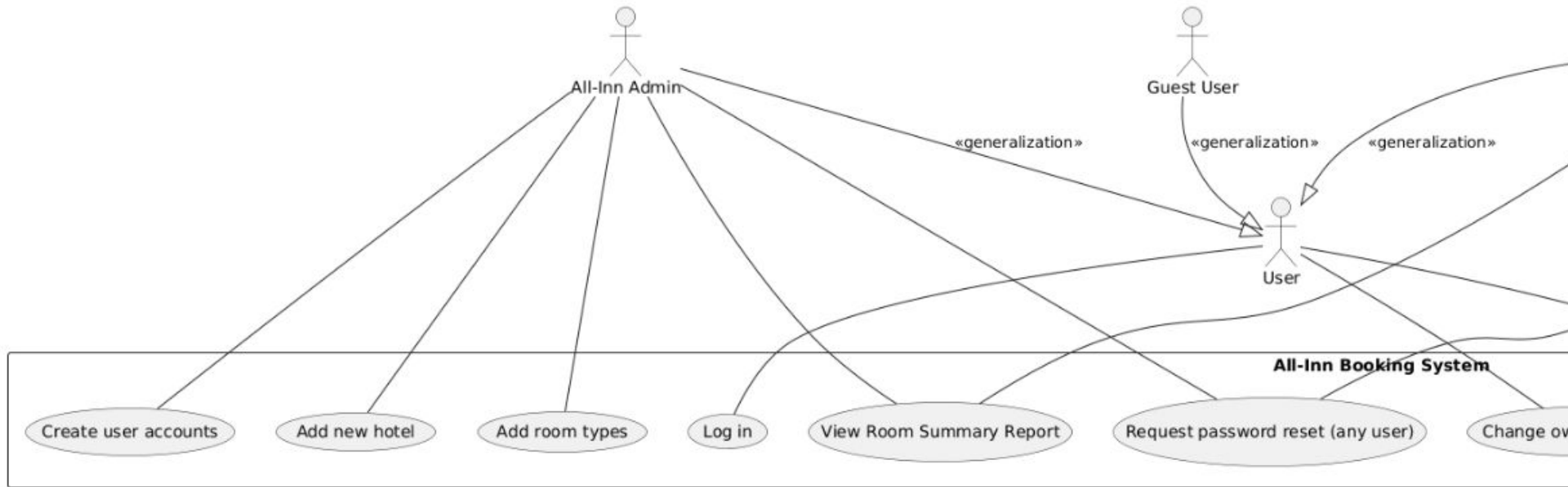


Hotel Reservation Use Case: Zoom-In Left

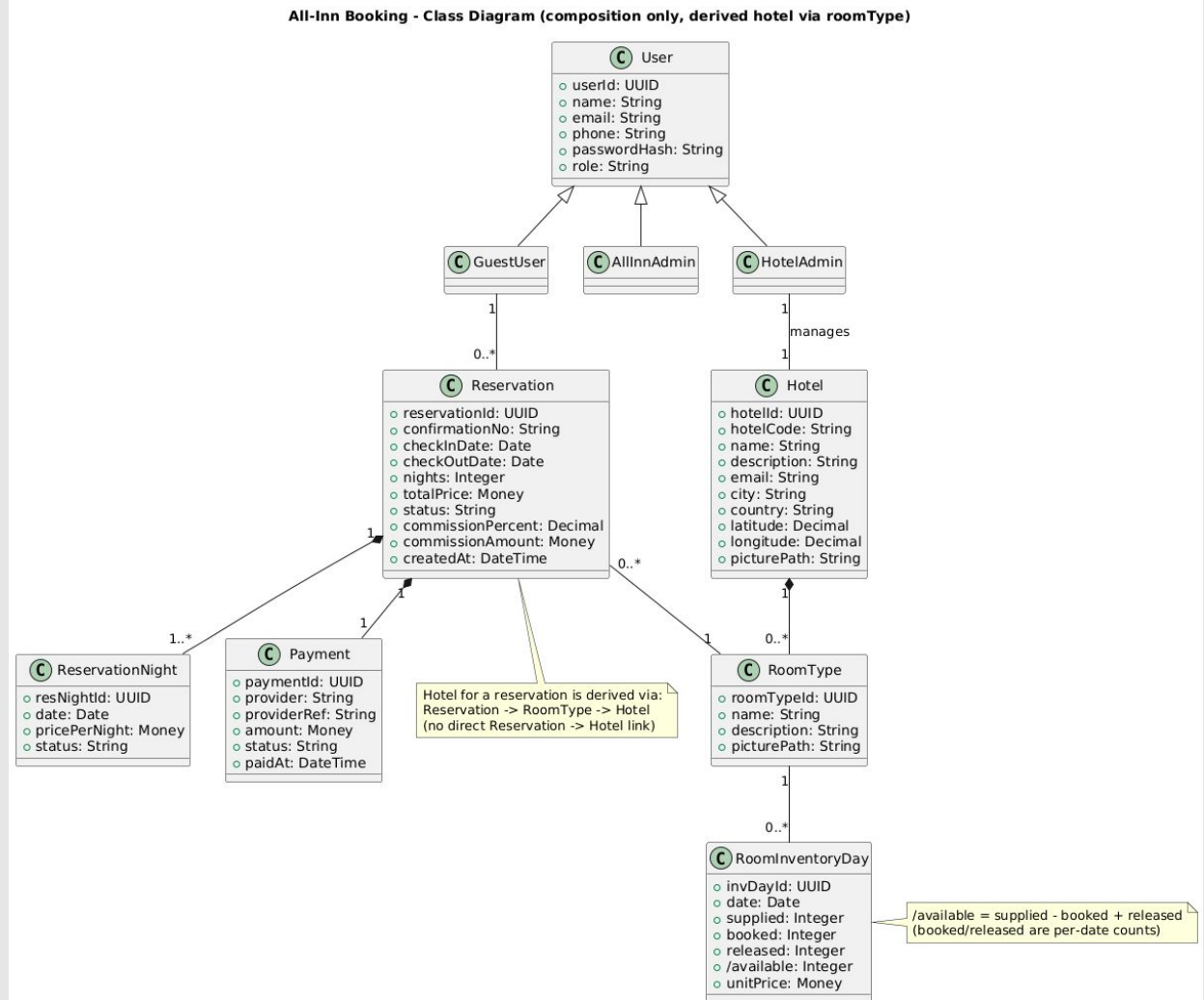


Zoom-in Left

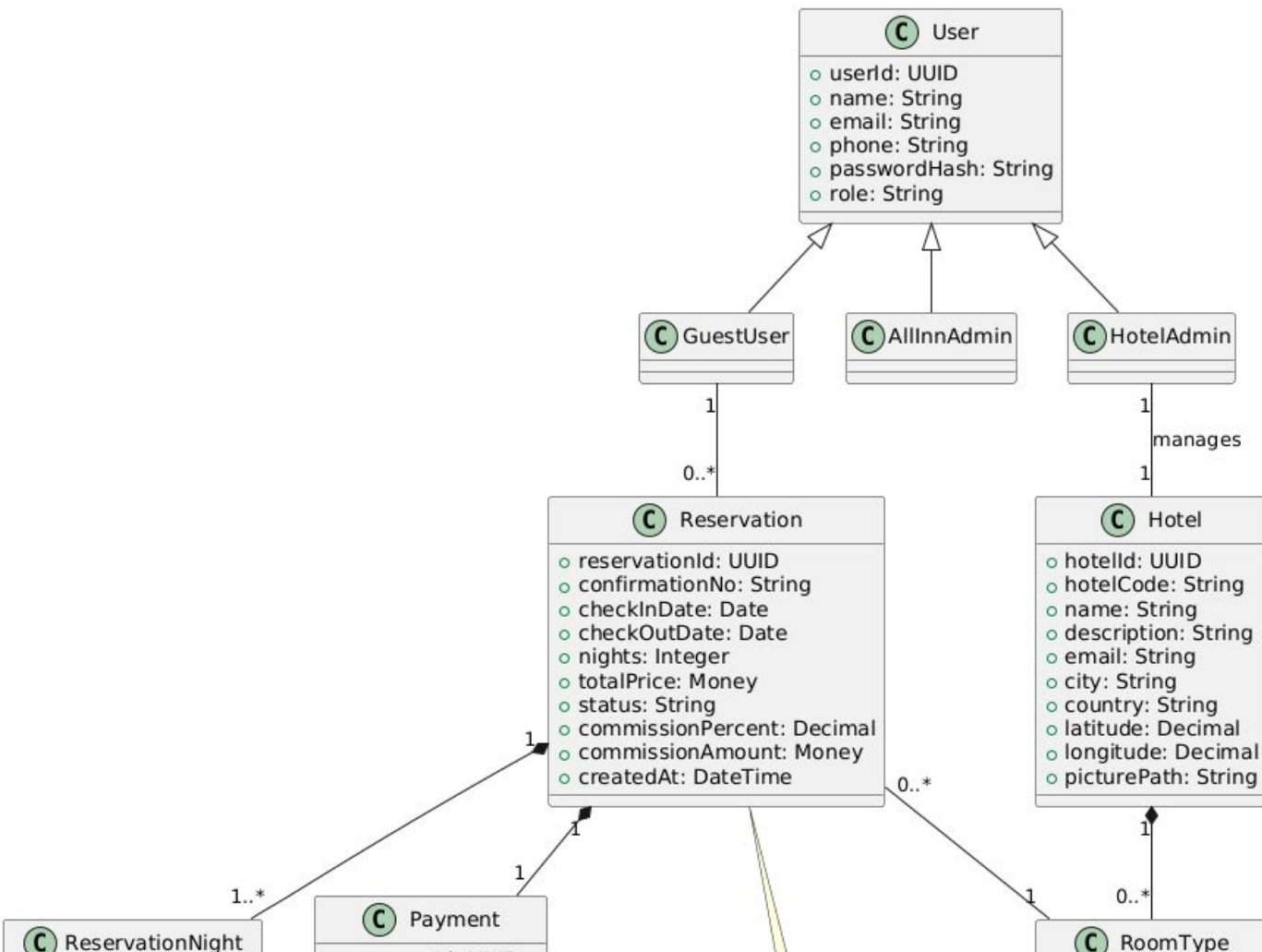
All-Inn Booking - User/Admin Use Cases



Hotel Class Diagram



All-Inn Booking - Class Diagram (composition only, derived hotel via roomType)



Activity Diagram

Sample Exam Problem: In a main workflow: the Guest searches hotels given a city with date range, views room prices in that hotel, makes a booking of 1 or more rooms, pays via K-Bank, system confirms a booking. Payment may be denied, where the guest can try again until she succeeds or decides not to book. Draw an **activity diagram** .

Activity Diagram

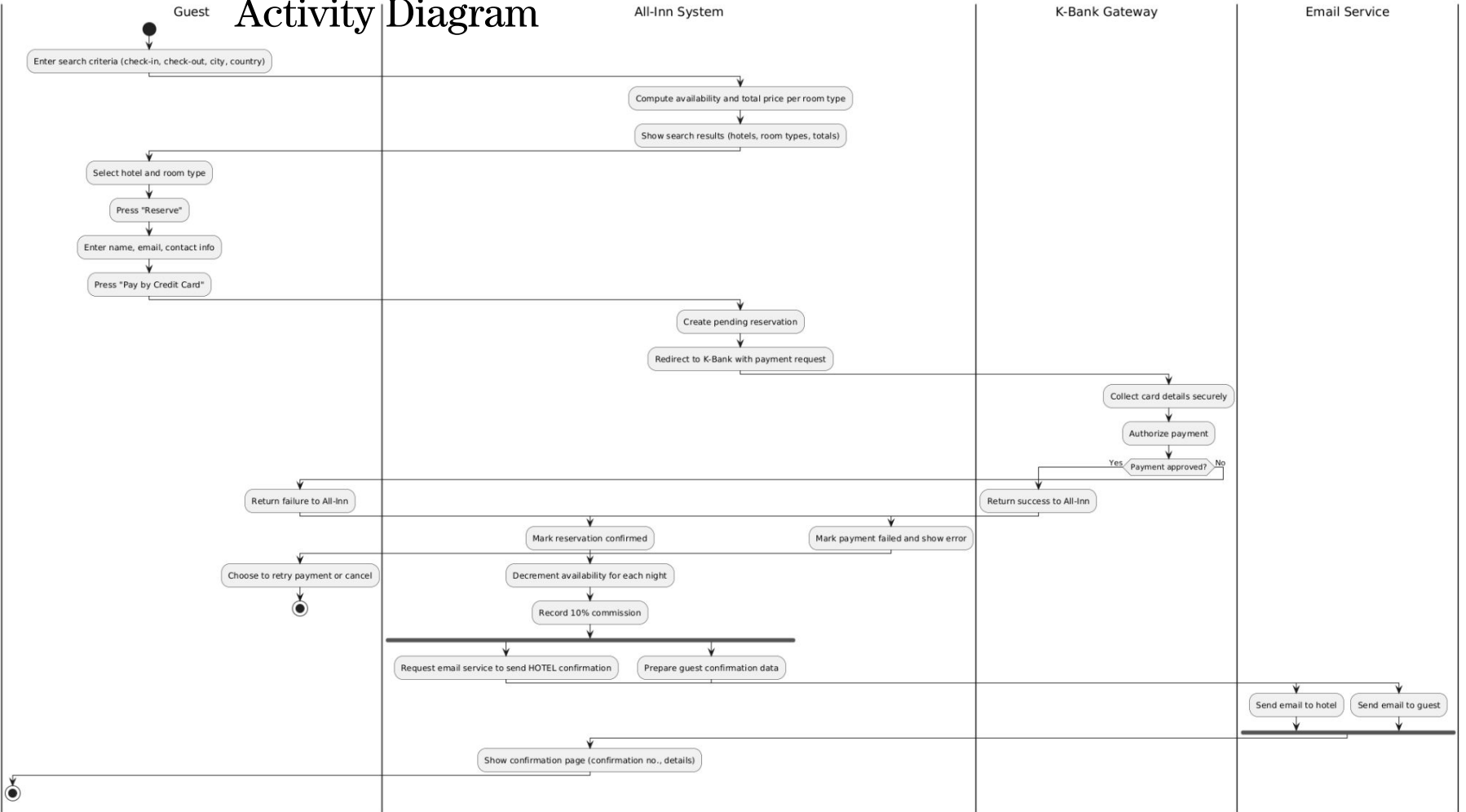
All-Inn Booking - Main Purchase Workflow (Activity Diagram)

Guest

All-Inn System

K-Bank Gateway

Email Service



All-Inn Booking - Main Purchase Workflow (Activity Diagram)

Guest

All-Inn System



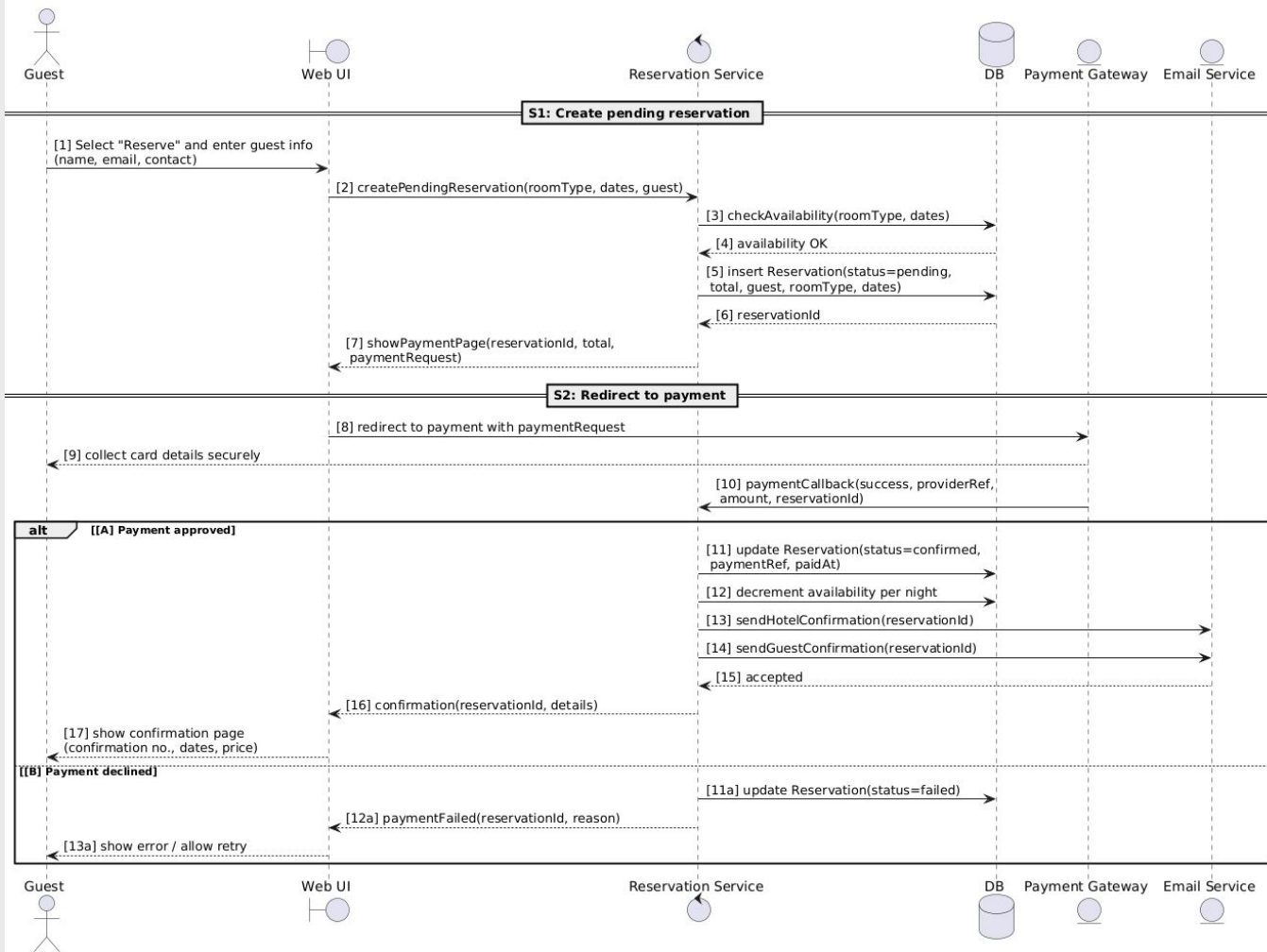
Sequence Diagram

Example Exam Problems:

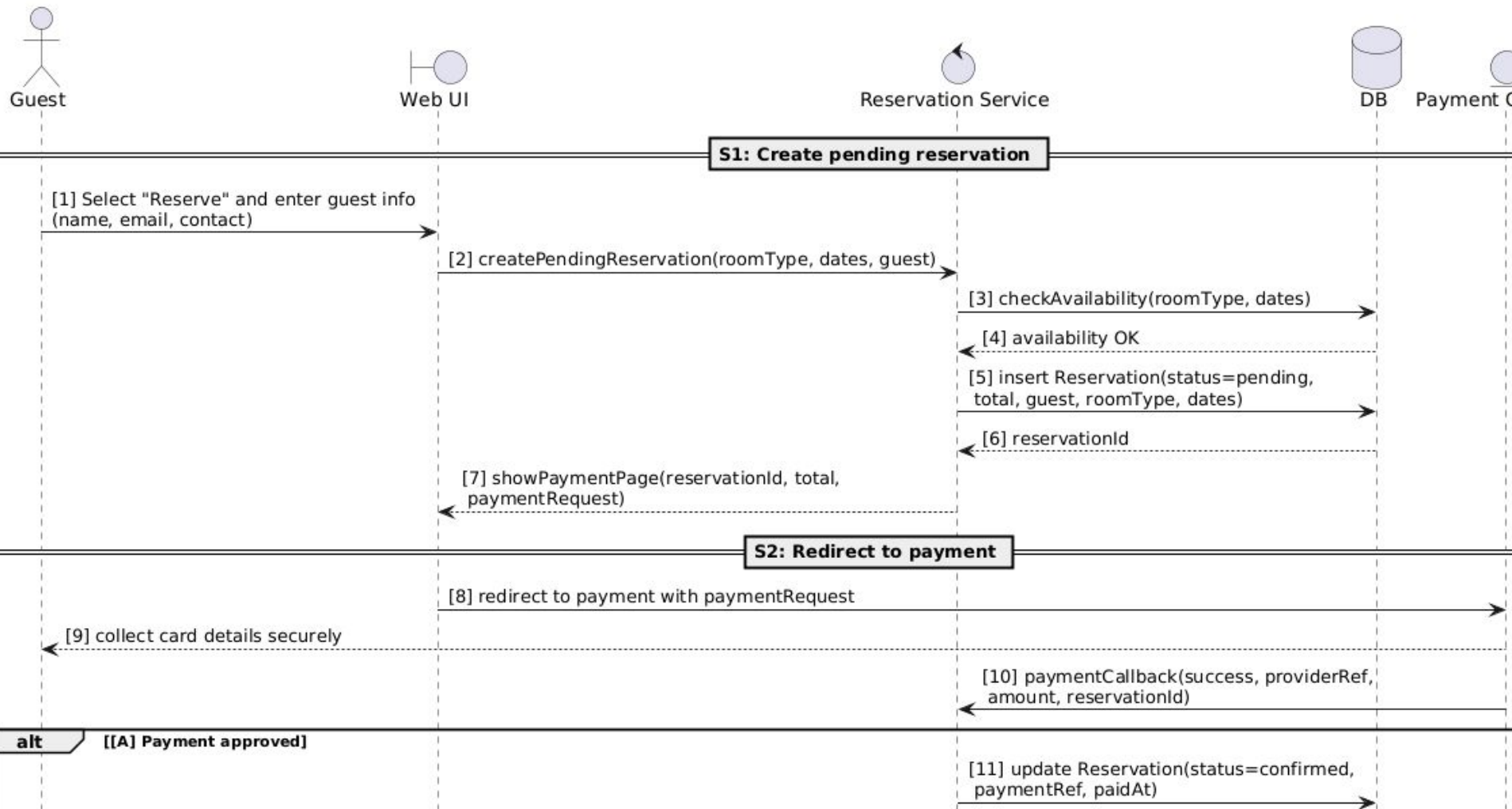
- A. Show the sequence for scenario when Guest reserves a room and successfully pays with a credit card.
- B. Show the sequence for scenario when Guest reserves a room and has a declined payment, but allowed to try again and then succeeds with another card.

Who are the Actors? Answer: Guest, Web UI, Reservation Service, Payment Gateway, DB, Email Service.

All-Inn - Sequence: Guest reserves and pays (labeled)

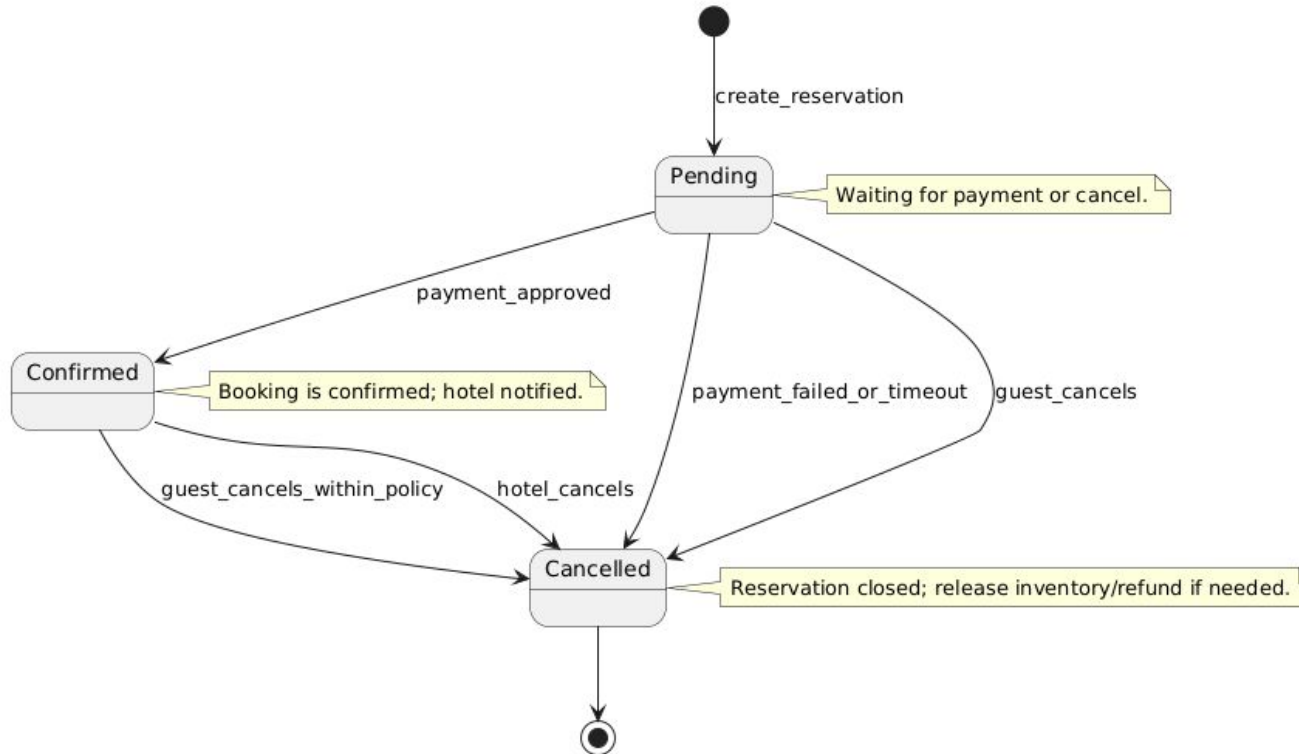


All-Inn - Sequence: Guest reserves and pays (labeled)



UML State Diagram

Reservation Lifecycle - State Diagram (ASCII-safe)



Sample Exam

Problem: Model the Reservation lifecycle using a UML State Diagram. Assume we have Pending state prior to the payment. Once payment made it is confirmed. Later a guest may decide to cancel to get a refund.

Software Service Ticket Example

Functional Requirements List

ABC Co. has hundreds of customers using its various software applications. ABC wants all customers to report change requests (includes bugs) by visiting the website http://www.ABC.co.th/service_ticket. Here are the requirements:

- Four types of users can Login: Customer - submits ticket, Change Control Engineer (CCE) - approves it for work by programmer, Programmer - works on the ticket, IT Admin - only for creating users etc. (not involved in ticket processing)
- After login the top of the page shows the person's name, user type (Customer / Change Control Engineer (CCE) / Programmer / IT Admin), and user's company.
- The Ticket users (Customer, CCE, and Programmer) see a list of all service tickets grouped by Software Name + Company sorted by due date. The Customer sees only their company's tickets. A user can pick a listed ticket to view the ticket and its actions list.
- There is a panel for the Ticket Users to create a new ticket or edit an existing ticket. Ticket fields: Ticket ID (auto number), Ticket Date, Program Name (list), the Form/Report/Component Name to change, Type of Problem (bug fix, missing feature, added work, other), Problem Description, Priority (high, regular, low), Requested by User Name, User Company Code, Request Date, Initial Due Date, Last Due Date, Last Status. Attachments: allow one picture (e.g., screen capture) and one file (doc/zip/etc.). These are kept in the network file system.
- Each ticket shows an Action Records History list with: Action Date, Done by User Name, user's role (customer / change control engineer / programmer), Action Taken Notes, New Due Date, New Service Status.

Functional Requirements List

- When a New Ticket is saved, one Action Record is automatically generated with the New Due Date set from Ticket's Initial Due Date, and New Service Status set to "Requested".
- After any action is saved, the Ticket's Last Status and Last Due Date attributes are updated using that latest action.
- New Actions Records can be added to the Ticket only by CCE and Programmers. Action records cannot be edited/deleted once saved.
- Change Control Engineers (CCE) review and approve requests can add actions to tickets that are "Requested". They can add an action for a ticket to set "Approved" (and optionally a New Due Date and Notes) or "Closed by CCE." They cannot modify the ticket.
- Programmers can add actions to tickets that are "Approved" or "In Progress".
 - They pick a ticket to work on, add an action record and keep Status as "In Progress," but add Notes and a New Due Date.
 - They may add more action records along the way (e.g., update due date or add Notes).
 - When finished, they add an action record with Status: "Closed by Programmer" and New Due Date: Completion date.
- A closed ticket (either "Closed by CCE" or "Closed by Programmer") cannot receive new actions.
- Other functions are done by IT Admin Users: Manage Company Info, Manage User Info (includes user type and company), Manage User Services, and Manage Software Info.

Service Ticket UI.

Ticket Form View Generic Design (Minimalist for Exam)

Service Ticket Form View Generic Design
User: Ying Kesuka Role: Customer

Tickets Users Software Company Logout Reports Authorization (App Controls)

[Close][New][Save][Clear][Print] (Form Controls)

Ticket No.: *Ticket Date: [Date]
Issued by User: *User Company: [LoV]
*For Software: [LoV] *Software Component:
*Problem Type: ▼ *Priority: ▼ {Low, Medium, High}
*Problem Description:
*Initial Due Date: [Date]
Attach Image: [Browse][Open] Attach File: [Browse][Open]
Last Due Date: Last Status:

List of Action Records: [New Action]

#	Action ID	Action Date	Action Done By	Role of User	Work Done Notes	New Due Date	New Status
1	AC25-01	15/09/25	Somchai	Customer	Started	20/09/25	Requested
2	AC25-01	16/09/25	Bam	CCE	Approved	20/09/25	Approved
3	AC25-01	18/09/25	Kate	Programmer	Started working on bug fix. Must move due date.	22/09/25	In-Progress

Sample Exam Problem: Show a simple UI for a Service Ticket Form View. It should have ticket information on the header and a list of action records for that ticket. Also show menu items for the core use cases in this application.

Also show the Tickets List View UI for Programmer Role.

Service Ticket UI.

Ticket List View For Programmer to Work On Ticket

Tickets List View

User: Ake Numchoke **Role:** Programmer

[Tickets](#) [Users](#) [Software](#) [Company](#) [Logout](#) [Reports](#) [Authorization \(App Controls\)](#)

[Close][New]

Filter Status: ☐ Requested ☒ Approved ☒ In-Progress ☐ Closed by CCE ☐ Closed by Programmer

- CCE will by default only see status: Requested. Programmers by default will see status: Approved and In-Progress. Customer sees all (limited to customer's company).

Search / Group / Filter: [OK]

Next Action: ☒ Open Form (Primary Key) ☐ Select Record (Foreign Key)

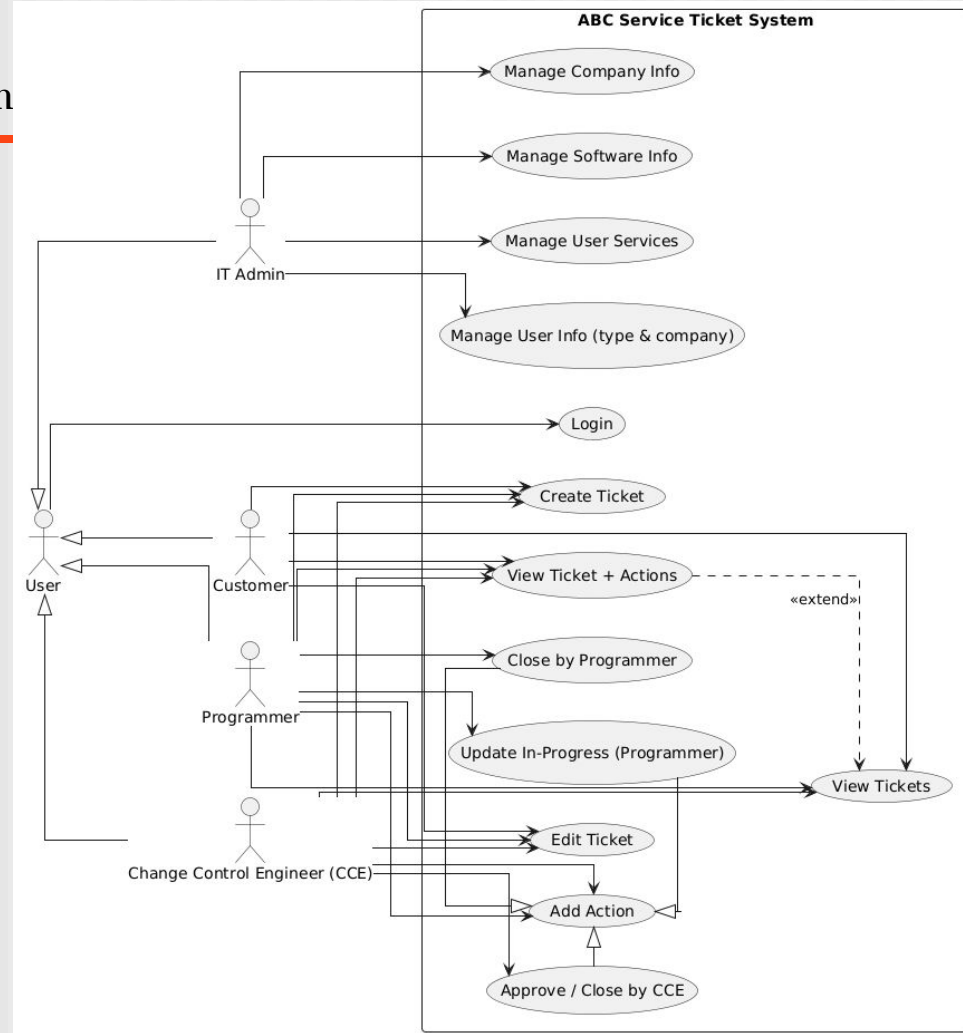
Click Row to select a record: (Will open form or select the record, depending on “Next Action”)

[illegible]

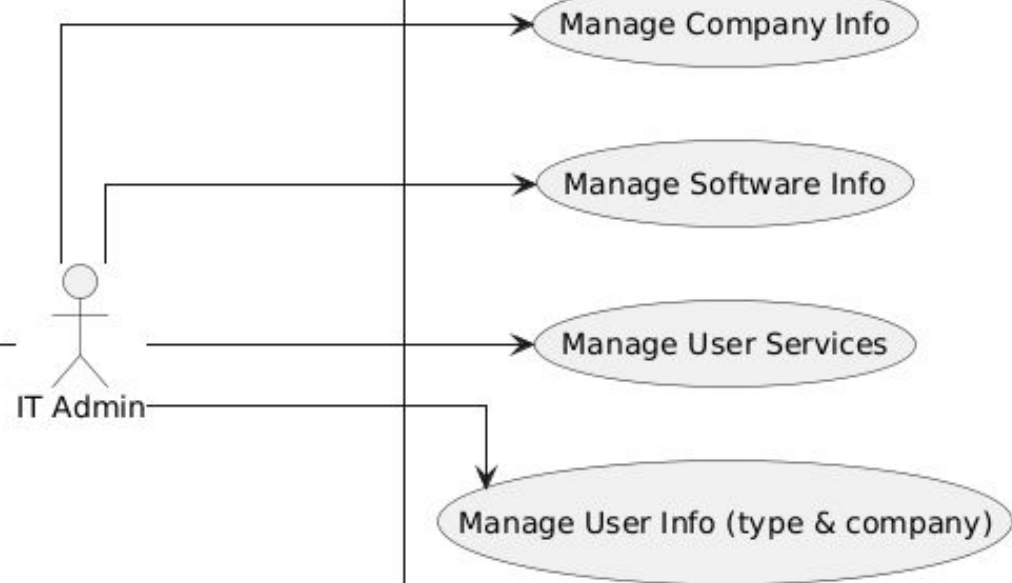
Ticket Use Case:

Sample Problem: Correct this diagram

1. All links from actor to use case should have no arrows.
2. Remove all links from actors to "View Ticket + Actions" as this is only accessible via "View Tickets" which it already extends.
3. Remove all actor links to "Create Ticket". It should extend "View Tickets".
4. Remove all actor links to "Edit Ticket". It should extend "View Tickets".
5. Remove all links to "Approve / Close by CCE". Add an <<extend>> link from it to "View Ticket + Actions".
6. Remove all links to "Update In-Progress (Programmer)". Rename it to "Update In-Progress / Close by Programmer". Add an <<extend>> link from it to "View Ticket + Actions".
7. Remove "Close by Programmer"



ABC Service Ticket System

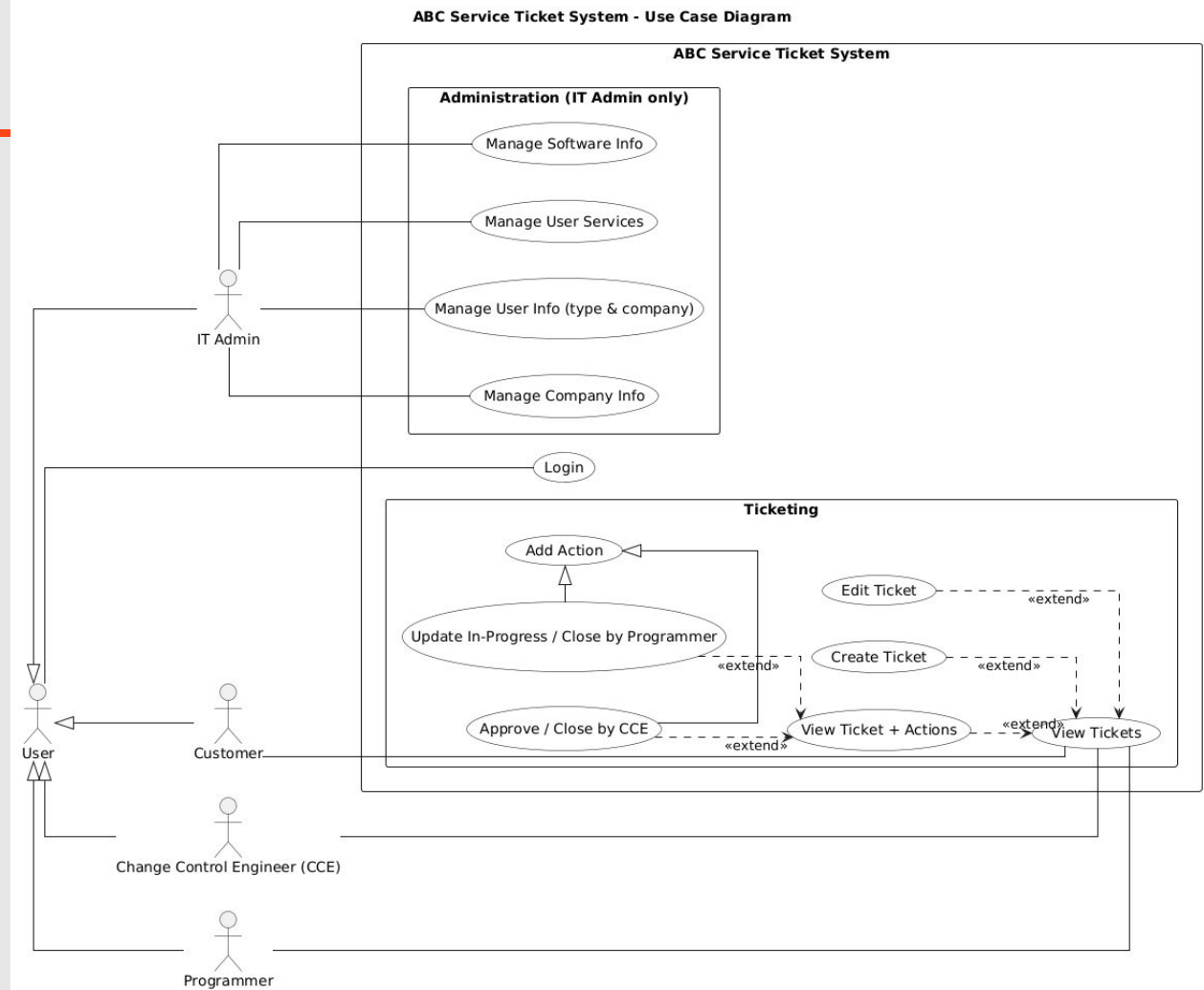


Login

Create Ticket

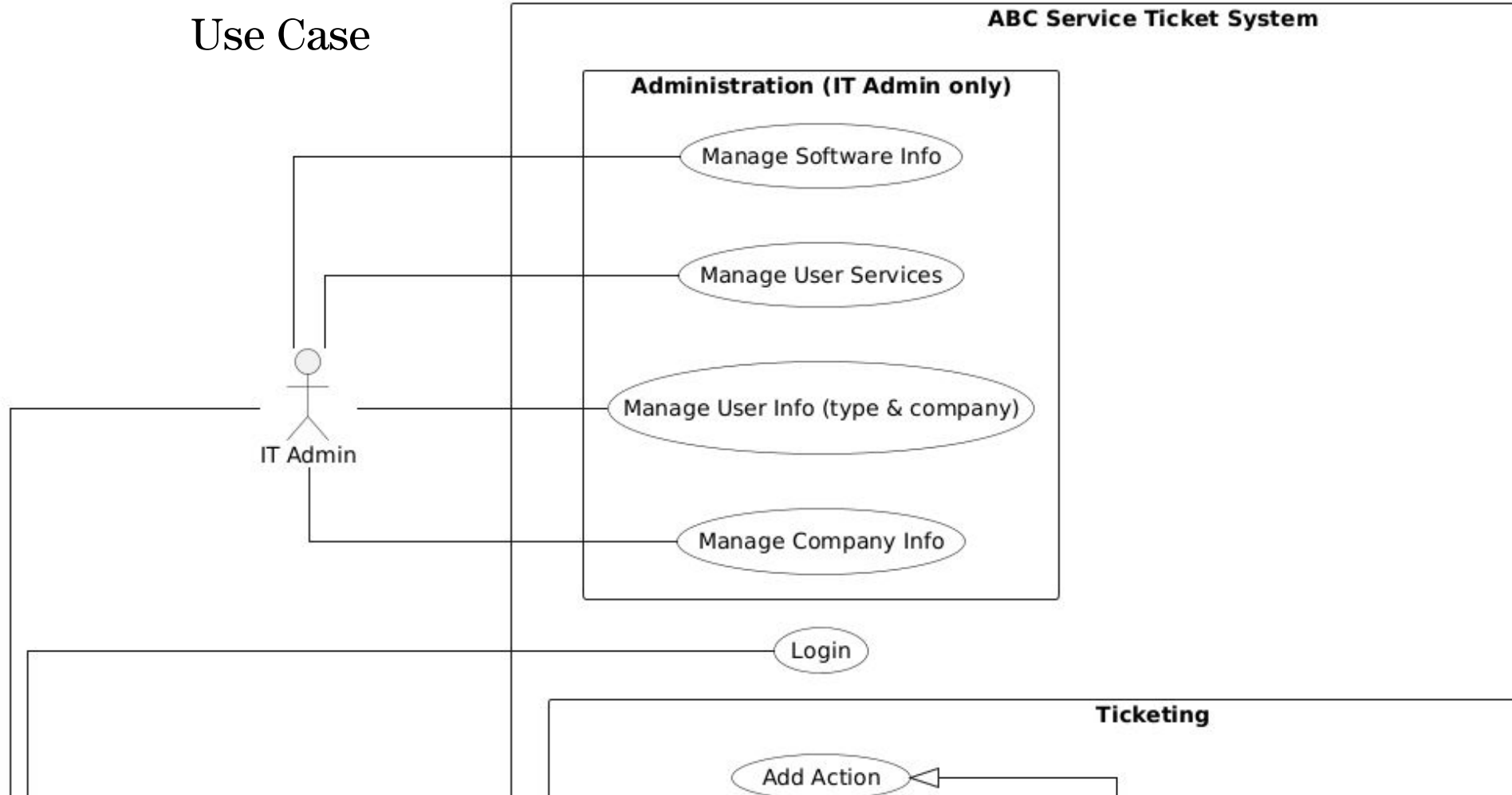
View Ticket + Actions

Corrected Use Case



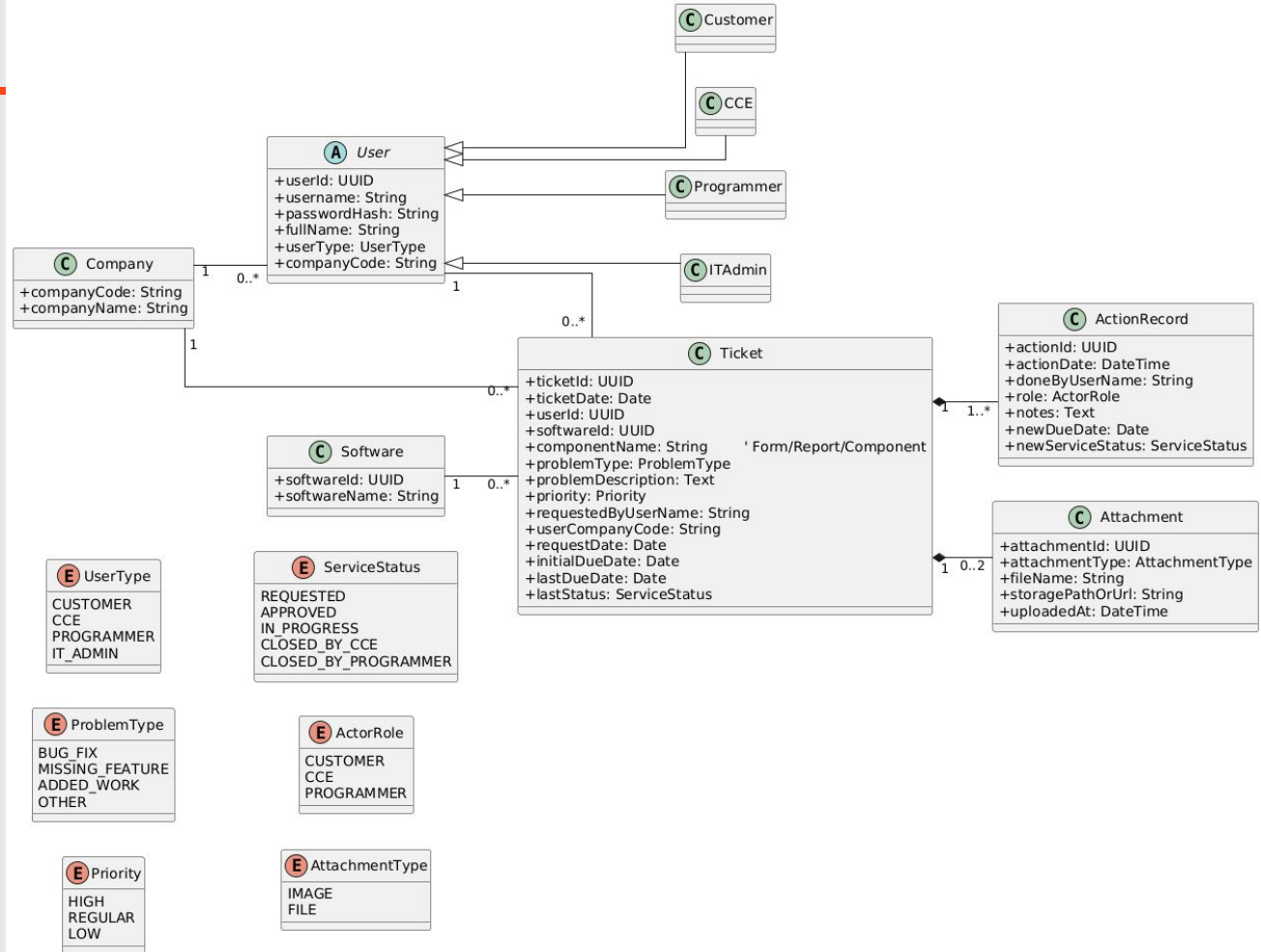
Corrected Use Case

ABC Service Ticket System - Use Case Diagram

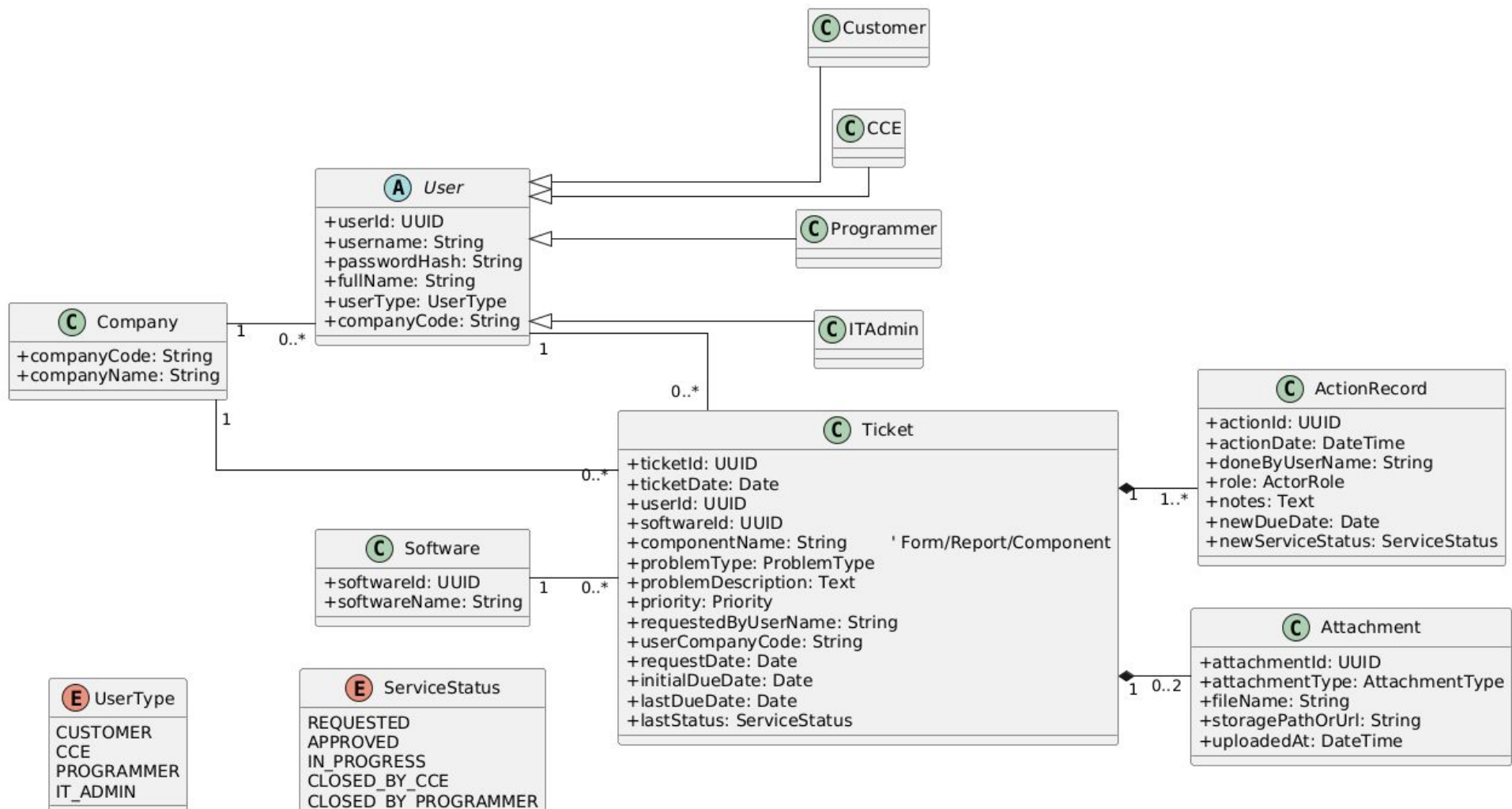


Ticket Class Diagram

ABC Service Ticket System - Class Diagram (Attributes Only, with Composition)



ABC Service Ticket System - Class Diagram (Attributes Only, with Composition)



Activity Diagram for Ticket Lifecycle Workflow

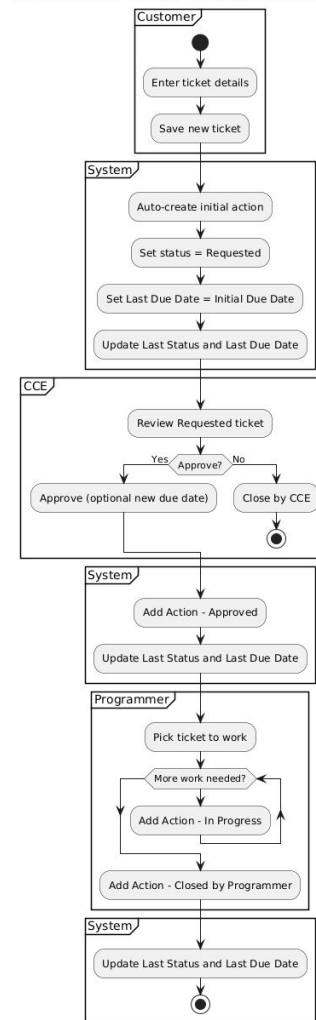
Sample Problem

Draw an **Activity Diagram** with swimlanes (Customer, CCE, Programmer, System) for the **ticket lifecycle** .

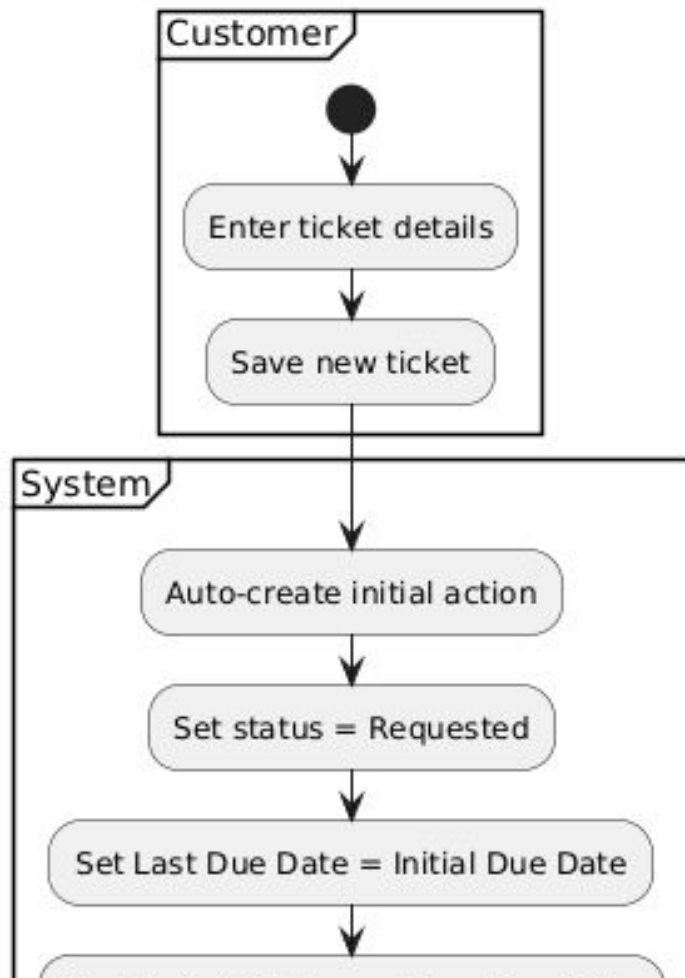
- Start with the **Customer** creating and saving a new ticket.
- The **System** auto-creates the initial action and sets the ticket status to *Requested*.
- The **CCE** reviews Requested tickets and either **approves** them (status → *Approved*) or **closes by CCE** (status → *Closed by CCE*).
- The **Programmer** works only on *Approved* tickets, adding *In Progress* actions until finishing with *Closed by Programmer*.
- After every action, the **System** updates the ticket's Last Status and Last Due Date.
- End when the ticket is *Closed*.

Activity for Ticket Lifecycle

Ticket Lifecycle - Activity Diagram (Swimlanes)



Ticket Lifecycle - Activity Diagram (Swimlanes)

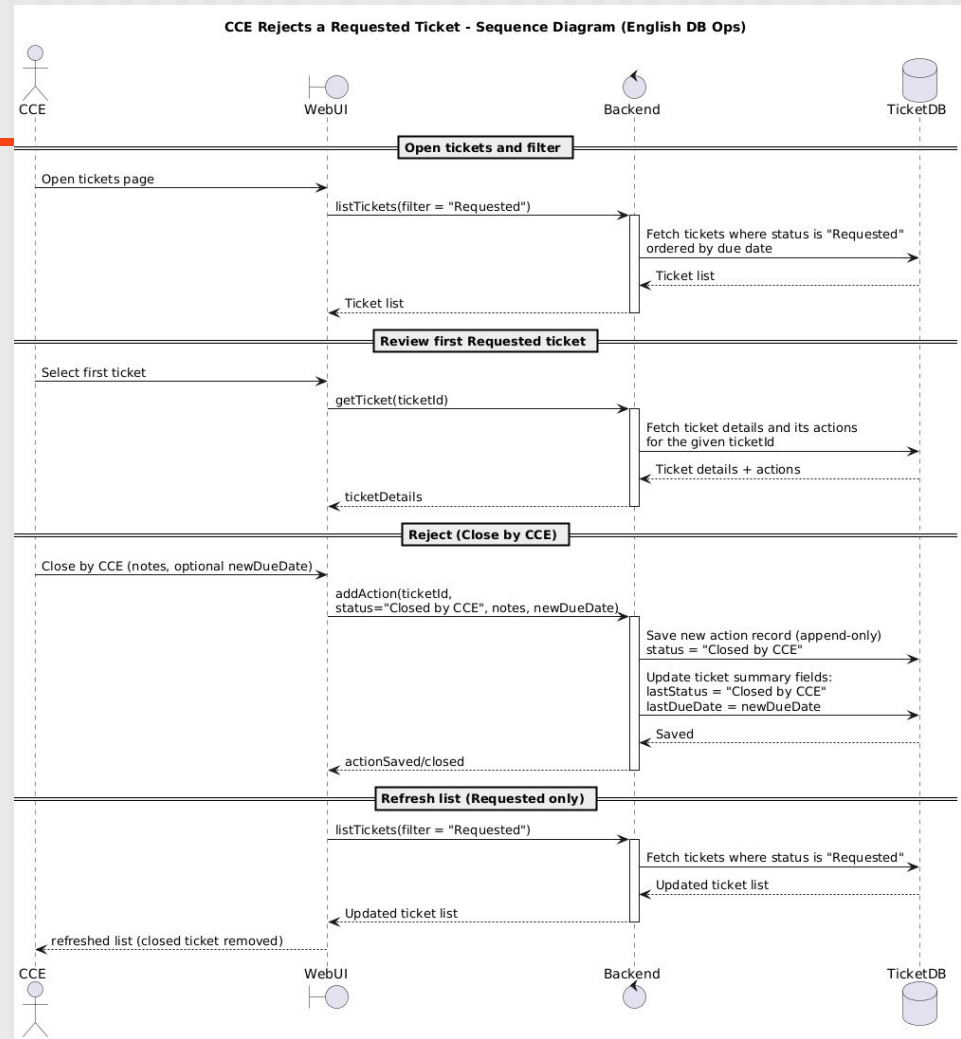


Sequence Diagram

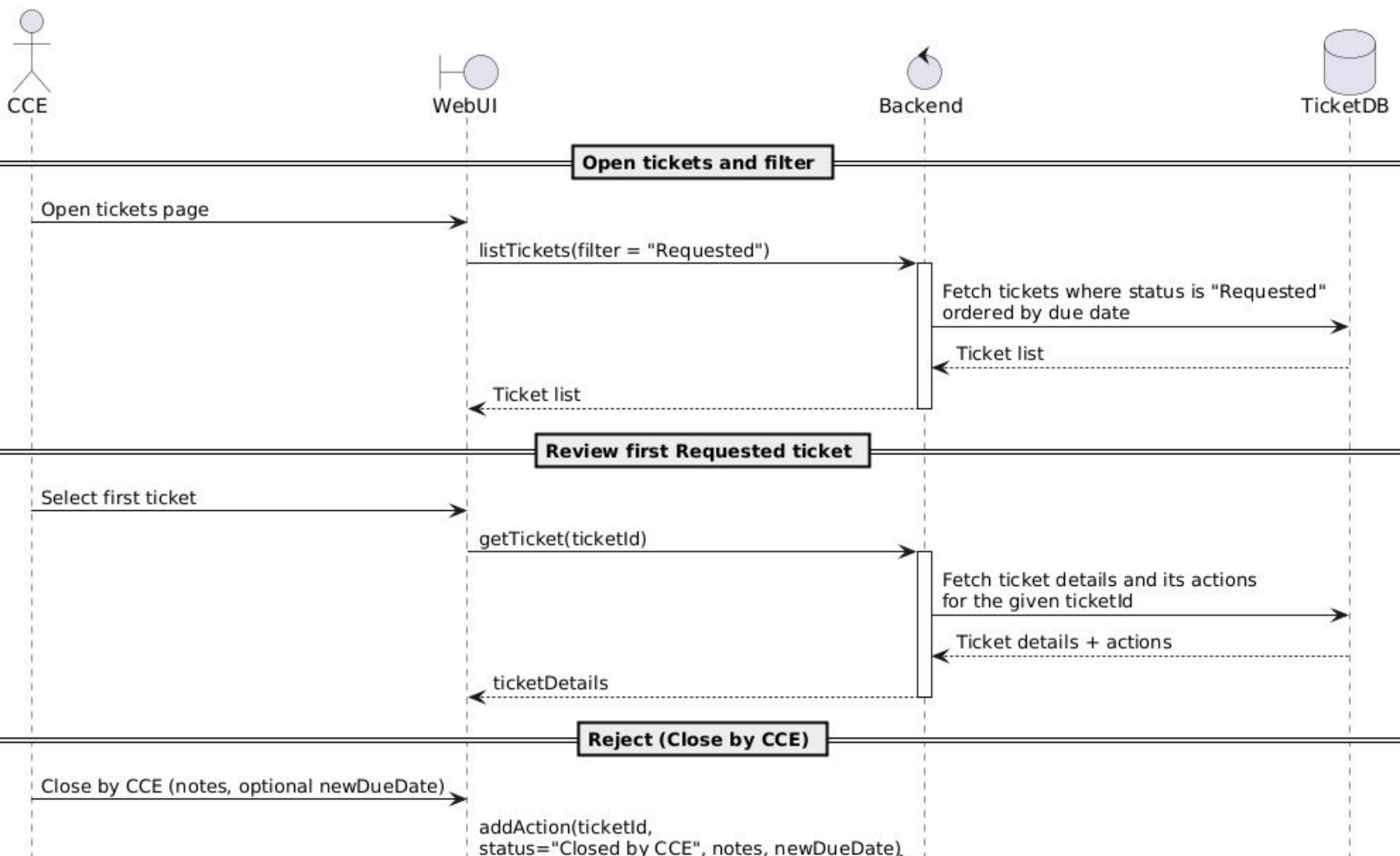
Sample Problem: Draw a Sequence Diagram for this scenario:

Assume the CCE is already authenticated. The CCE opens the tickets list view, filters tickets with status Requested, reviews the first such ticket, and rejects it (Close by CCE). The system makes the necessary changes as per requirements.

Sequence Diagram



CCE Rejects a Requested Ticket - Sequence Diagram (English DB Ops)



State Diagram

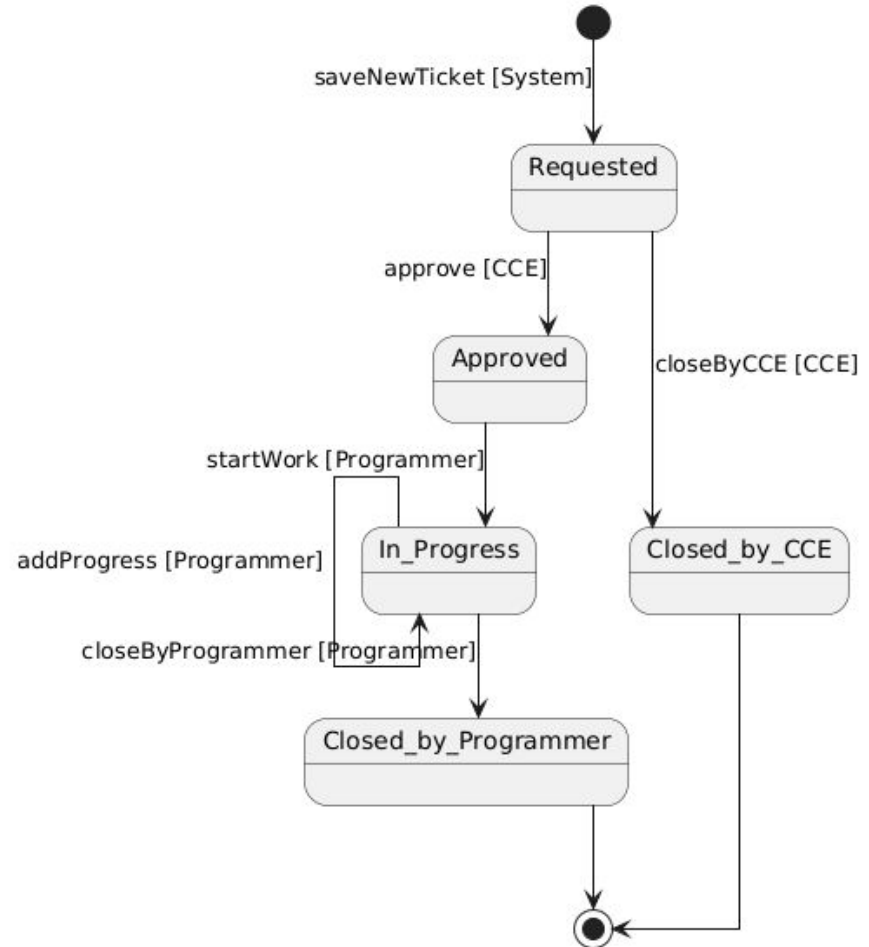
Sample Problem. Draw a **UML State Diagram** for a **Service Ticket** that models only its **status** change based on the requirements provided. Clearly show start and end states. Label each transition with the triggering event and (in brackets) the actor, e.g., **approve [CCE]**.

Solution:

- Initial state: when a ticket is saved, status becomes Requested (system sets this automatically).
- CCE actions (on Requested state only)
- Programmer actions on Approved and In Progress States
- Once closed (by CCE or Programmer), no further actions are allowed.

UML State Diagram

Ticket Status - UML State Diagram



University Grades Submission Example

For Krungthep University (KU)

Online Grades Submission Functional Requirements

Objective: Enable instructors at Krungthep University (KU) to submit student grades online instead of using paper forms.

Key Functions for Instructors

1. **Secure Login**
 - Instructors log in with their KU account.
 - Only valid accounts can access the system.
2. **View Courses Offered**
 - Instructors can see the list of courses and sections offered in their department for a chosen semester.
3. **View Registered Students**
 - For a selected course and section(s), instructors can see which students are registered.
 - This list can be brought directly into the online grade sheet.
4. **Maintain Course Grade Sheet**
 - Each grade sheet has a header (Course, Semester) and a list of registered students.
 - Instructors enter or update the grade for each student. Initially it is in draft mode.
 - Instructors may also print a copy of the grade sheet for their own records.

Online Grades Submission Functional Requirements

5. Submit Grades

- Once all grades are entered, the instructor changes from the sheet from **Draft** mode to **Ready** mode.
- Instructor can then submit the grade sheet (can only submit if in **Ready** mode).
- On submission:
 - Student transcripts are updated automatically.
 - Each student is notified of their grade by email.
- The system confirms that submission was successful.
- The grade sheet status is now in **Submitted** mode.

External Systems:

- **KU Single Sign-On:** Used to verify instructor login and manage passwords.
- **KU Registration System:** Supplies lists of offered courses and registered students.
- **KU Transcript System:** Records the official grades for each student.
- **KU Student Information System:** Provides student contact details (e.g., email).
- **KU's Email Service via Google Workspace:** Sends grade notifications to students.

Grade Submission UI.

Course Grade Sheet List View Generic Design (Minimalist for Exam)

Course List View Generic Design

User: Ying Kesuka Role: Instructor

Select Course Reports Logout

[Close]

For Semester: [LoV] - Default will be current semester

Owner Department: [LoV]

Search:

[FILTER][CLEAR]

Select a Row to View/Enter/Edit Grades for that Course-Section:

Course ID	Course Code	Course Name	Section	Semester	Grades Status
1933	CPE100	Intro Programming	1	2025-1	Draft
1935	CPE100	Intro Programming	2	2025-1	-
2115	CPE211	Hands on Computing	1	2025-1	Draft
2116	CPE211	Hands on Computing	2	2025-1	Submitted
2117	CPE211	Hands on Computing	3	2025-1	Ready
2118	CPE211	Hands on Computing	4	2025-1	-

Sample Exam Problem: Show Courses List View UI. It allows users to select the semester and department. The list of courses then appears for user to view/enter/edit grades for that course. Each Course is: Course Code + Section + Semester.

Grade Submission UI.

Course Grade Sheet Form View Generic Design (Minimalist for Exam)

Sample Exam

Problem: Show Grades Submit Form View for the instructors to enter the total score and grade for each student for a course (code + section + semester). Assume the course is already selected from the Course List View before coming to this form view.

Course Grade Submit Sheet Form View Generic Design

User: Ying Kesuka Role: Instructor

[Submit Grades](#) [View Courses](#) [Logout](#) [Reports](#)

[Close][Save][Clear] (Form Controls)

Grade Sheet No.:

Entry Date: [Date]

Submitted by User:

*User Role:

For Semester:

*For Course-Section:

Number Students:

Current State: [Change to Ready]

Grades Submitted Date: (autoset after Submit: Draft to Ready to Submit)

Instructor Remarks:

Enter Score and Grades for Students in this Course/Section:

#	Student ID	Student Name	Degree Program	Registration Type	Total Score	Grade
1	680112	Ake Amhit	BE. Computer Engineering	Regular (A to F)	97.00	_A_
2	680191	Bruno Baabin	BE. Computer Engineering	S/U	36.00	_U_
3	681012	Cindi Saetan	BS. Computer Science	Regular (A to F)	82.00	_B_
4	680133	Direk Dumrong	BE. Computer Engineering	Regular (A to F)	76.00	_C_
5	680201	Ellie Esterfan	BE. Computer Engineering	Regular (A to F)	51.00	_F_

Group Work

Group Project and Group Labs 5-7

Group Work Specs File

The screenshot shows a Google Docs interface for a document titled "group_work_specs_v1.DOCX". The document content is as follows:

CPE 334 Group Work Specification

Introduction to Software Engineering in the Age of AI Agents
Semester 1/2026 | Draft for instructor review | Issued: 6 October 2026

1. Purpose

Work in a team of two to plan, engineer, build, test, deploy, maintain, and release a full-stack software product in a new problem domain. Apply the SDD, TDD, Git, testing, and review practices learned through the individual TokTickIT labs 1-4. Use AI agents to assist with engineering work while remaining responsible for its correctness.

The project is one continuous development effort with three project submissions and three group labs performed on the same application.

Assessment	What it demonstrates
Project Deliverable 1	You can plan the product and engineer/build Sprint 1.
Project Deliverable 2	You continued the engineering discipline through Sprint 2 and into Sprint 3.
Final Delivery	You can finish, integrate, verify, and release the complete product.

Group Work Includes

Assessment	What it demonstrates
Project Deliverable 1	You can plan the product and engineer/build Sprint 1.
Project Deliverable 2	You continued the engineering discipline through Sprint 2 and into Sprint 3.
Final Delivery	You can finish, integrate, verify, and release the complete product.
Lab 5	Can you ship it? Containerize the application, establish CI/CD, and deploy it.
Lab 6	Can somebody else verify it? Obtain independent UAT and manage defects.
Lab 7	Can you change it without breaking it? Analyze and implement a change, run regression tests, and assess release readiness.

Points Distribution

Category / assessment	Raw points	Course Grade %
Individual Labs 1–4, existing aggregate	400	20%
Group Lab 5: CI/CD, Docker, and GCP deployment	100	5%
Group Lab 6: Cross-team UAT and defect management	100	5%
Group Lab 7: Maintenance, metrics, and release readiness	100	5%
All labs combined	700	35%
Project Deliverable 1	80	8%
Project Deliverable 2	70	7%
Final Project Delivery	100	10%
Group project subtotal	250	25%

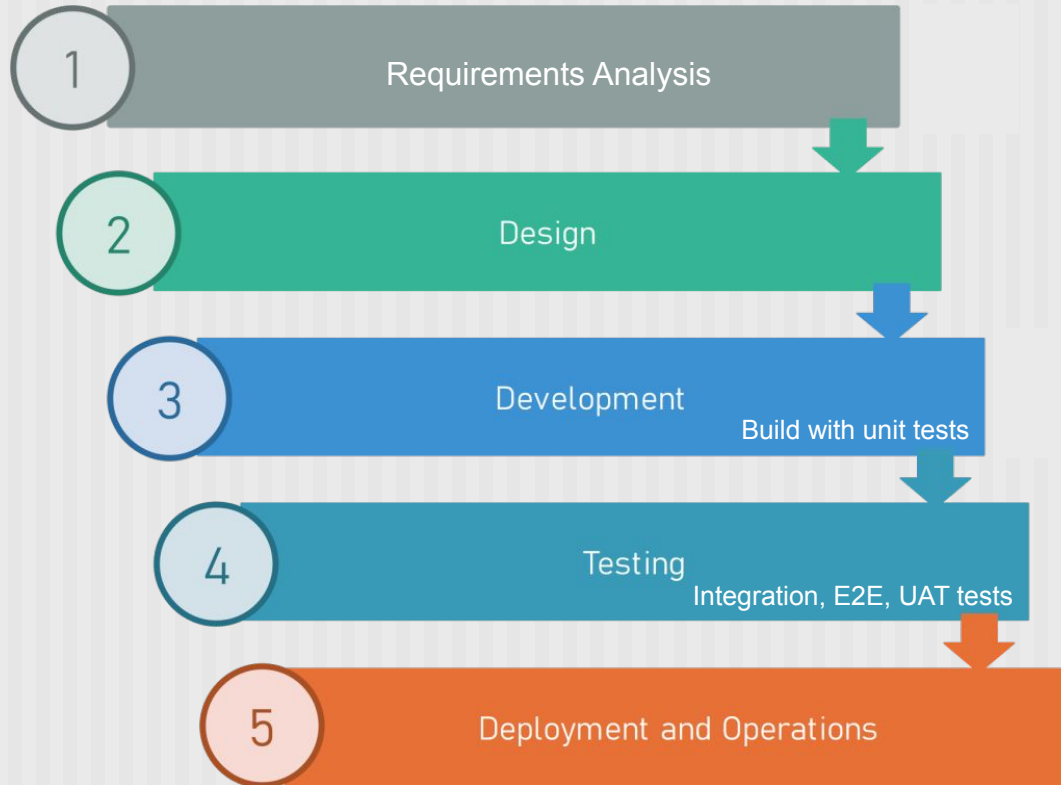
Each group work deliverable has its **own spec sheet** :

- Project Deliverables 1, 2, Final
- Lab 5, Lab 6, Lab 7

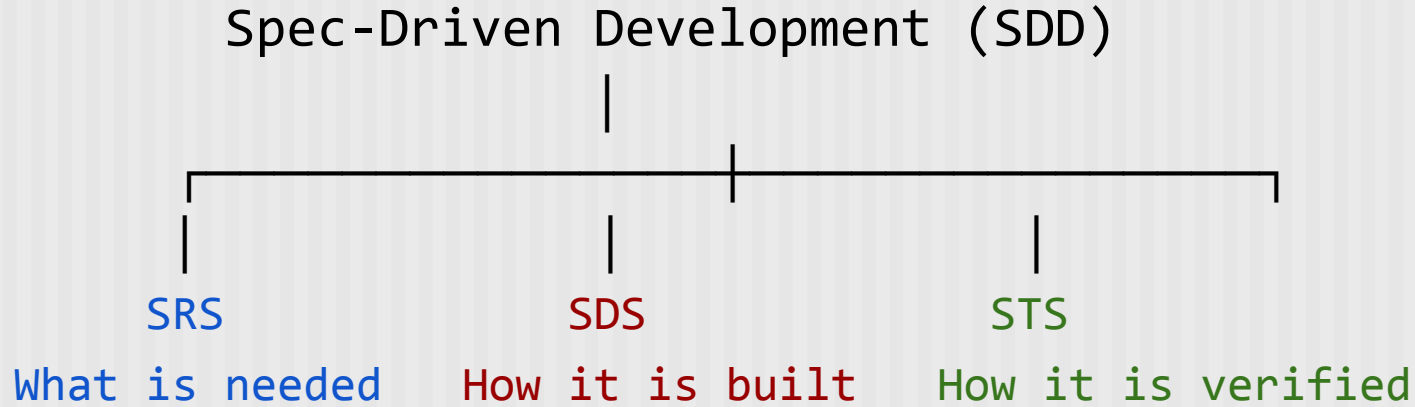
Deliverables and Schedule

Date	Activity / milestone	Points
6, 8 Oct	Miterm Exam Review Project launch: topics, scope, team workflow, engineering baseline	
12 - 25 Oct	Exam period; plan around your examination commitments	
26 Oct - 1 Nov	Midterm break; independent project work	
3, 5 Nov	Workshop: SRS, system SDS, Sprint 1 contract, tests, integration Lab 5: CI/CD, Docker, GCP deployment.	
8 Nov Sunday	Project Deliverable 1 due: baseline + working Sprint 1	80 project
10, 12 Nov	Lab 6: cross-team UAT and defect management	
15 Nov Sunday	Lab 5 due: deployed Sprint 1 and repeatable delivery	100 lab
17, 19 Nov	Lab 7: maintenance, metrics, release readiness	
22 Nov Sunday	Lab 6 due	100 lab
24, 26 Nov	Final Exam Review. Project Deliverable 2 Discussion.	
29 Nov Sunday	Deliverable 2 due Lab 7 due	70 project 100 lab
1 - 11 Dec	Final examination period	
13 Dec Sunday	Final product, report, and demo video due	100 project

Anatomy of a Software Project



Spec-Driven Development (SDD) for Coding Agents



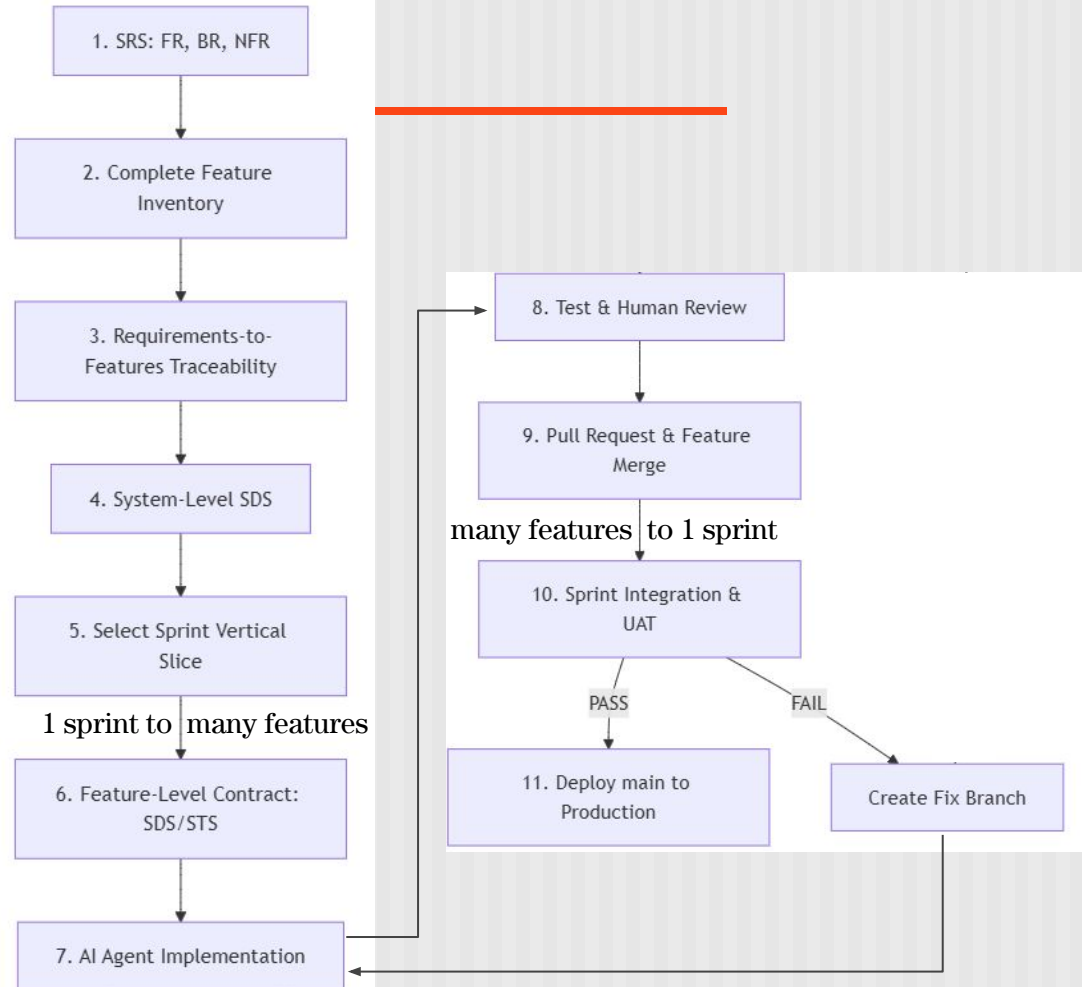
SRS - software requirements specification

SDS - Software Design Specification

STS - Software Test Specification

Spec Driven Dev

SDD



Recommended Design Process

1. **SRS.** FR, BR, NFR.



2. **Complete Feature Inventory.** Feature-A, Feature-B, Feature-C, ...



3. **Requirements-to-Features Traceability.** Map every FR, BR, and NFR to features. FR-003 → Feature-E.



4. **System-Level SDS.** Architecture, tech stack, UI standards, API conventions, security, deployment



5. **Select the Sprint Vertical Slice.** Choose a “Feature Bundle” to be implemented in the sprint



6. **Feature-Level Engineering Contract:** SDS + UI/API/Data Specs + Acceptance Criteria + STS



7. **AI Agent Implementation.** Implement one bounded Issue at a time according to the specifications



8. **Test and Human Review.** Run automated tests, inspect the code and Git diff, correct defects.



9. **Pull Request and Integration.** Review, approve, and merge feature branch into the Sprint staging branch



10. **Sprint Integration, System Testing, and UAT.** Run tests on Sprint staging branch. If PASS: Pull Request from Sprint staging → main. If FAIL: create a fix branch on Sprint staging branch, update the SDS/STS if needed, perform Steps 7-9, and repeat Step 10.



11. **Per Release: Deploy main Branch to Production**

per feature

1. The Software Requirements Specification (SRS) Document

$$\text{SRS} = \text{BR} + \text{FR} + \text{NFR} + \text{supporting context}$$

The requirement catalogue

BR Business goals and outcomes

FR Functions and system behavior

NFR Quality attributes and constraints

These requirements form the core of the SRS.

Purpose & scope

Business context, objectives, system boundary, and exclusions.

Actors & interfaces

Users, external systems, inputs, outputs, and interactions.

Assumptions & constraints

Dependencies, operating conditions, policies, and technical limits.

Acceptance & traceability

Testable criteria and links from requirements to design and tests.

Requirements List (minimalist)

1. **Serve customers at the counter**
 - Cashiers can start a checkout, add or remove items, apply valid discounts, and see the running total.
 - A sale can be **cancelled** any time before it is finished.
2. **Accept payments**
 - Customers can pay **by cash** or **by credit card** which is processed via a **Payment Service Provider** (eg., K-Bank)
 - The system **works with the card machine** from, say, K-Bank to send the amount and receive approval
 - If a card is **declined**, the cashier can retry, switch to cash, or cancel the sale.
3. **Print documents: Receipt** for successful sale, **Refund Receipt** for refunds. Prints to store **receipt printer**.
4. Updates stock in **store's inventory service** app: **reduces** stock of the sold item, adds back stock of returned items.
5. **Handle returns and refunds.**
 - Cashiers or managers can find the original sale and **process a return**.
 - The system **refunds** the customer (cash or card, following store policy) and records the reason.
6. **Manager Functions**
 - Managers can view a **daily sales summary** for a selected date, including total cash & card amount received.
 - Managers can **add or update products and prices** to be used by POS.

SRS Prompt Session Example



<https://docs.google.com/document/d/1Y87HZ0RwSAUjcm8pPtiZ-s5-iOuZmo7L3uZXpuleovI/edit?usp=sharing>

For this chat, assume I'm a software engineering student wanting your help in creating good SE artifacts such as Requirements Specifications, Design Specs, Test Scenarios, etc.

I have these requirements provided to me for a POS system by my instructor. I have to iteratively refine it until they become numbered FR, BR, and NFR. First, please read this and ask me questions about parts of this POS system that may need more clarification before you can draft the requirements lists.

1. **Serve customers at the counter**
 - Cashiers can start a checkout, add or remove items, apply valid discounts, and see the running total.
 - A sale can be **cancelled** any time before it is finished.
2. **Accept payments**
 - Customers can pay **by cash** or **by credit card** which is processed via a **Payment Service Provider** (eg., K-Bank)
 - The system **works with the card machine** from, say, K-Bank to send the amount and receive approval
 - If a card is **declined**, the cashier can retry, switch to cash, or cancel the sale.
3. **Print documents: Receipt** for successful sale**, Refund Receipt** for refunds. Prints to store **receipt printer**.
4. Updates stock in **store's inventory service** app**: reduces** stock of the sold item, adds back stock of returned items.
5. **Handle returns and refunds. **
 - Cashiers or managers can find the original sale and **process a return**.
 - The system **refunds** the customer (cash or card, following store policy) and records the reason.
6. **Manager Functions**
 1. Managers can view a **daily sales summary** for a selected date, including total cash & card amount received.
 2. Managers can **add or update products and prices** to be used by POS.

Recommended Design Process

1. **SRS.** FR, BR, NFR.



2. **Complete Feature Inventory.** Feature-A, Feature-B, Feature-C, ...



3. **Requirements-to-Features Traceability.** Map every FR, BR, and NFR to features. FR-003 → Feature-E.



4. **System-Level SDS.** Architecture, tech stack, UI standards, API conventions, security, deployment



5. **Select the Sprint Vertical Slice.** Choose a “Feature Bundle” to be implemented in the sprint



6. **Feature-Level Engineering Contract:** SDS + UI/API/Data Specs + Acceptance Criteria + STS



7. **AI Agent Implementation.** Implement one bounded Issue at a time according to the specifications



8. **Test and Human Review.** Run automated tests, inspect the code and Git diff, correct defects.



9. **Pull Request and Integration.** Review, approve, and merge feature branch into the Sprint staging branch



10. **Sprint Integration, System Testing, and UAT.** Run tests on Sprint staging branch. If PASS: Pull Request from Sprint staging → main. If FAIL: create a fix branch on Sprint staging branch, update the SDS/STS if needed, perform Steps 7-9, and repeat Step 10.



11. **Per Release: Deploy main Branch to Production**

per feature

2. Complete Feature Inventory: POS

ID	Feature	Feature scope
Feature-A	User Authentication and Access Control	Sign-in, user roles, function permissions, and store-based access restrictions for Cashiers, Managers, Administrators, and Head-Office Managers.
Feature-B	Store and Terminal Operating Context	Identification of the current store and POS terminal, association of transactions with that context, and support for multiple stores and terminals.
Feature-C	Product Catalog and Price Management	Addition and modification of products, product information, VAT-inclusive selling prices, and manager access restrictions.
Feature-D	Sale Lifecycle Management	Starting a sale, maintaining its current state, cancelling it before completion, and preventing modification after completion.
Feature-E	Cart and Sale-Item Management	Adding products, changing quantities, removing products, and recalculating the sale when its items change.
Feature-F	VAT-Inclusive Pricing and Total Calculation	Calculation and display of item amounts, subtotal, Thai VAT information, discount, and final sale amount using Thai baht and satang.

2. Complete Feature Inventory: POS

ID	Feature		Feature scope
Feature-G	Order-Level Management	Discount	Discount Percentage and Discount Amount entry, percentage-to-amount calculation, satang rounding, manual discount-amount handling, validation, and recalculation when the cart changes.
Feature-H	Payment Selection	Method	Selection of either cash or credit card, enforcement of the no-split-payment rule, and permitted switching of payment methods.
Feature-I	Cash Payment Processing		Entry of cash tendered, validation that sufficient cash was received, change calculation, and cash-sale completion.
Feature-J	Card Authorization	Payment	Creation of a unique payment-attempt reference, durable recording of the attempt, communication with the card machine and Payment Service Provider, and handling of approved or declined results.
Feature-K	Card Payment Recovery and Reconciliation		Handling communication interruptions, unresolved payment results, approved payments that could not be finalized, PSP status queries, duplicate-charge prevention, and recovery of affected sales.
Feature-L	Sale Finalization and Durable Recording		Validation of successful payment, assignment of a unique sale identifier, permanent recording of the sale, and atomic creation of the associated pending inventory update.

2. Complete Feature Inventory: POS

ID	Feature	Feature scope
Feature-M	Receipt Printing and Reprinting	Printing a receipt after successful sale completion, reporting printer failures, and reprinting without duplicating or reversing the sale.
Feature-N	Inventory Update Outbox	Creation and durable storage of a pending inventory-update record containing the sale, store, terminal, date and time, product, and quantity information.
Feature-O	Inventory Message Delivery	Asynchronous message transmission, acknowledgement processing, retained-message retry, and duplicate inventory-update prevention through unique transaction identifiers.
Feature-P	Operational Exception Monitoring	Viewing unresolved card payments, payment-recovery cases, pending inventory messages, and inventory messages that have repeatedly failed delivery.
Feature-Q	Daily Sales Reporting	Selection of a business date, assigned-store reporting for Managers, selected-store reporting for Administrators and Head-Office Managers, consolidated reporting, and separate cash and card totals.
Feature-R	Audit Trail	Recording the responsible user, store, terminal, date, and time for completed sales, cancellations, discounts, card-payment recovery, and product or price changes.

3. Requirements-to-features traceability

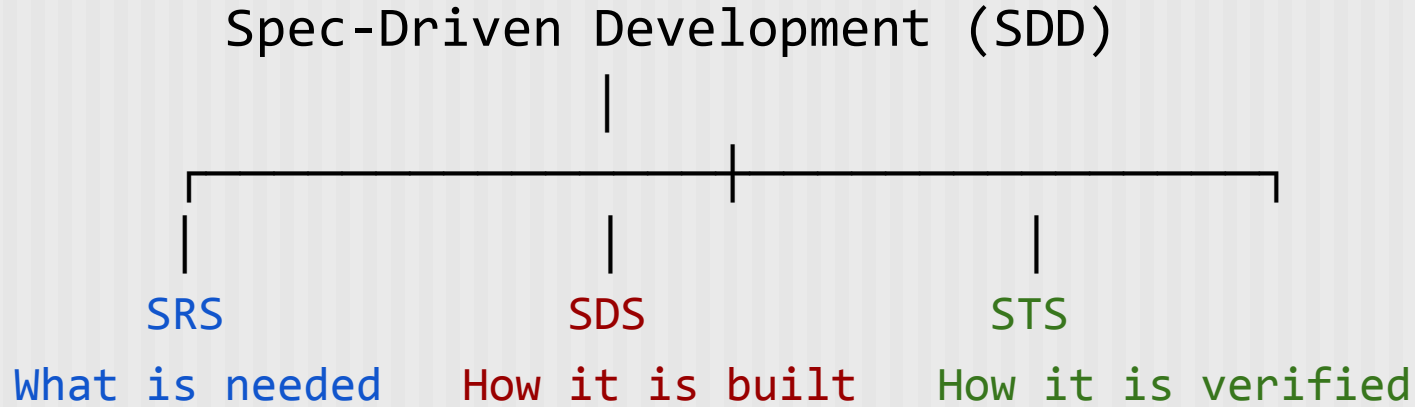
<https://docs.google.com/document/d/1Y87HZ0RwSAUjcm8pPtiZ-s5-iOuZmo7L3uZXpuleovI/edit?usp=sharing>

POS Requirements-to-Features Traceability Matrix

1. Functional Requirements

Requirement	Requirement statement	Covered by feature(s)
FR-001	The system shall require each user to sign in under an authorized role, including Cashier, Manager, Administrator, or Head-Office Manager.	Feature-A: User Authentication and Access Control
FR-002	The system shall allow an authorized cashier to start a new sale.	Feature-A: User Authentication and Access Control; Feature-D: Sale Lifecycle Management
FR-003	The system shall allow the cashier to add products, change quantities, and remove products before the sale is completed.	Feature-E: Cart and Sale-Item Management
FR-004	The system shall display and update the subtotal, discount, VAT information, and final amount whenever the contents of the sale change.	Feature-F: VAT-Inclusive Pricing and Total Calculation; Feature-G: Order-Level Discount Management

Spec-Driven Development (SDD) for Coding Agents



SRS - software requirements specification

SDS - Software Design Specification

STS - Software Test Specification

4. Contents of SDS Document: System-Level

I. **System-Level SDS.** This defines shared technical decisions that apply across the application:

- System architecture
- Technology stack
- Major components and modules
- Overall data model
- Authentication and authorization approach
- UI Design System Specification
- API conventions
- Error-handling conventions
- Security, performance, logging, and other cross-cutting NFRs

Contents of SDS Document: Feature Level

II. For **Each Feature** , create a specification containing:

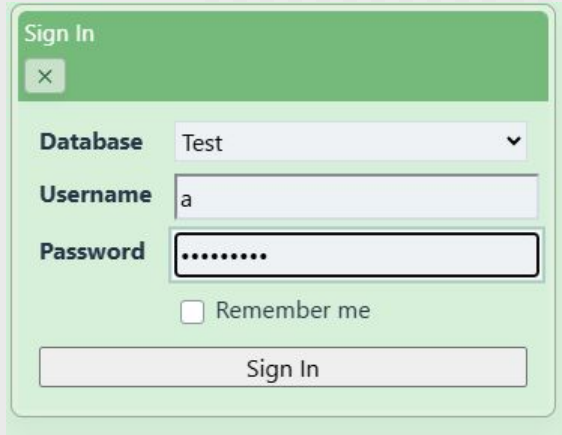
1. Feature purpose and scope
2. Related SRS requirements
3. Workflow and state transitions
4. Business rules
5. Data model changes
6. API endpoints
7. UI behavior
8. Permissions
9. Validation and error handling
10. Acceptance criteria
11. Test considerations

UI Design Specifications

- Theme and color palette
- Semantic color usage, such as success, warning, error, and information
- Typography and heading hierarchy
- Spacing and layout grid
- Buttons, forms, tables, cards, and other shared components
- Icons and imagery
- Responsive breakpoints
- Accessibility requirements, including contrast and keyboard navigation
- Light/dark mode, if supported
- Standard loading, empty, confirmation, and error states

Document	Example
SRS/NFR	NFR-UI-01: The application shall conform to the company's corporate branding guidelines.
SRS/NFR	NFR-UI-02: All text and interactive controls shall meet WCAG 2.1 AA contrast requirements.
SDS	Primary color: #0057B8; secondary color: #F5A623; error color: #D32F2F.
SDS	Primary buttons use the corporate blue background with white text.
SDS	Headings use the corporate typeface; body text uses Arial as the fallback.

Login Screen



Sign In

Database Test

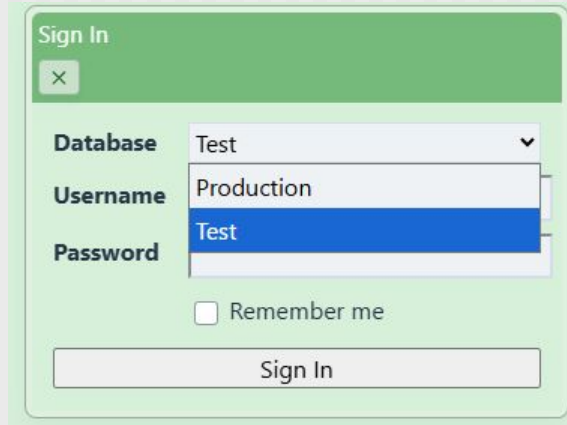
Username a

Password

☐ Remember me

Sign In

Can choose which database to login. For example, Production, Staging, Feature A Dev.



Sign In

Database Test

Username Production

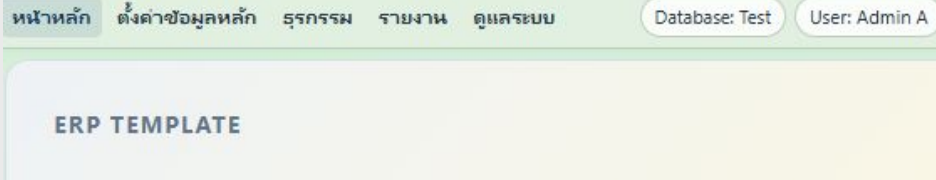
Password Test

☐ Remember me

Sign In

App Shell and Control Menu

Main Page | Master | Transactions | Reports | Security



Customer

Supplier

ตั้งค่าข้อมูลหลัก
ลูกค้า
สินค้า

Invoice

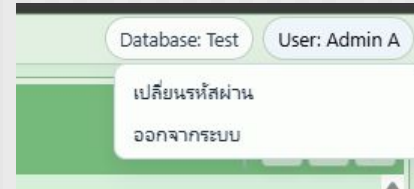
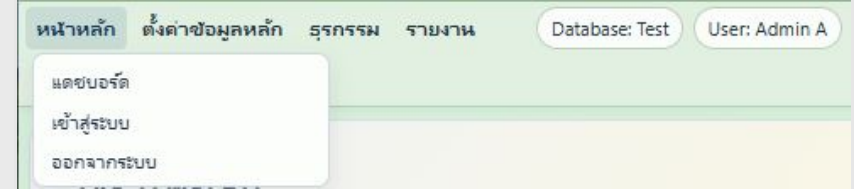
ธุรกรรม
ใบแจ้งหนี้

Invoice List

Products Sold

รายงาน ดูแลระบบ
รายการใบแจ้งหนี้
รายงานสินค้าที่ขาย

Main Page | Master | Transactions | Reports



Change Password

Logout

ดูแลระบบ
ผู้ใช้งาน
บทบาทของผู้ใช้งาน

Users

User Role

Invoice Model List View

หน้าหลัก ตั้งค่าข้อมูลหลัก **ธุรกรรม** รายงาน ดูแลระบบ Database: Test User: Admin A

ใบแจ้งหนี้ Invoice

เลขที่ใบแจ้งหนี้	No.	Date	Cus Code	Customer Name	Province	Country	Goods	ภาษีมูลค่าเพิ่ม
มีคำว่า...	วว/ดด/ปปปป หรือ:	มีคำว่า...	มีคำว่า...	มีคำว่า...	มีคำว่า...	มีคำว่า...	100 หรือ 10..20	100 หรือ 10..
SINV26-000001	30/04/2026	abc	Abc Co.		Thailand		18.00	
SINV26-000002	30/04/2026	abc	Abc Co.		Thailand		155.00	

Page Size 50 First Prev 1 Next Last

Line Item UI Specs

1. Line items always in a named tab.

2. Has button to add row to bottom on upper right side of grid. Button as tooltip.

3. Has buttons add row below this line, add row above this line, delete this line on both the left and right sides of each grid row, each with tooltips.

4. Has ordered item number to preserve ordering.

5. In edit mode the selected row's editable fields use whitish color; non-editable fields creamish. Shows text, checkbox, and drop-down lists.

6. Has LoV button as a separate column. Same for Date Picker Button.

7. Required fields have leading red "**".

8. When in View mode rows of the grid show in alternating light/dark shades.

9. Validation errors show as red box with tooltip-style error messages.

No.	* รหัสสินค้า Code	ชื่อสินค้า Prod. Name	* ราคาต่อหน่วย Qty	* จำนวน Units	Total	รวมเงิน
1	P-PEN-BLK	Ballpoint Pen Black	12.00	1.00 Piece	12.00	
2	P-STP-MINI	Mini Stapler	85.00	1.00 Piece	85.00	

No.	* รหัสสินค้า Code	ชื่อสินค้า Prod. Name	* ราคาต่อหน่วย Qty	* จำนวน Units	Total	รวมเงิน
1	P-PEN-BLK	Ballpoint Pen Black	12.00	1.00 Piece	12.00	
2	P-STP-MINI	Mini Stapler	85.00	1.00 Piece	85.00	
3	P-CLIP-M	Paper Clip Medium	15.00	3.00 Box	45.00	

No.	* รหัสสินค้า Code	ชื่อสินค้า Prod. Name	* ราคาต่อหน่วย Qty	* จำนวน Units	Total	รวมเงิน
1	P-PEN-BLK	Ballpoint Pen Black		1.		
2	P-STP-MINI	Mini Stapler		1.		

Unit Price must be over 0.00

Invoice form showing responsive behavior

Note the adjustments of attribute-value header fields while line item fields have a left-right scroll.

ใบแจ้งหนี้ Edit Mode Existing Status

×

+

✓

↺

Running ID

2

เลขที่ใบแจ้งหนี้ Invoice No.

SINV26-000002

*วันที่เอกสาร Invoice Date

30/04/2026

*รหัสลูกค้า Customer Code

abc

ชื่อลูกค้า Customer Name

Abc Co.

ที่อยู่บรรทัด 1 Address Line 1

ที่อยู่บรรทัด 2 Address Line 2

ที่อยู่บรรทัด 3 Address Line 3

ใบแจ้งหนี้ Edit Mode Existing Status

×

+

✓

↺

จังหวัด Province

ประเทศ Country

Thailand

มูลค่าสินค้า Goods Val

155.00

VAT %

7.00

ภาษีมูลค่าเพิ่ม VAT

10.85

ยอดชำระ Amt. Due

165.85

รายการสินค้า

	#	*รหัสสินค้า Code	ชื่อสินค้า Prod. Name	*ราคาต่อ
	1	P-NBK-A5	Notebook A5	
	2	P-PEN-BLK	Ballpoint Pen Black	

Recommended Design Process

1. **SRS.** FR, BR, NFR.



2. **Complete Feature Inventory.** Feature-A, Feature-B, Feature-C, ...



3. **Requirements-to-Features Traceability.** Map every FR, BR, and NFR to features. FR-003 → Feature-E.



4. **System-Level SDS.** Architecture, tech stack, UI standards, API conventions, security, deployment



5. **Select the Sprint Vertical Slice.** Choose a “Feature Bundle” to be implemented in the sprint



6. **Feature-Level Engineering Contract:** SDS + UI/API/Data Specs + Acceptance Criteria + STS



7. **AI Agent Implementation.** Implement one bounded Issue at a time according to the specifications



8. **Test and Human Review.** Run automated tests, inspect the code and Git diff, correct defects.



9. **Pull Request and Integration.** Review, approve, and merge feature branch into the Sprint staging branch



10. **Sprint Integration, System Testing, and UAT.** Run tests on Sprint staging branch. If PASS: Pull Request from Sprint staging → main. If FAIL: create a fix branch on Sprint staging branch, update the SDS/STS if needed, perform Steps 7-9, and repeat Step 10.



11. **Per Release: Deploy main Branch to Production**

per feature

4. System-level SDS design



Kaching System Level SDS v0.1.docx

https://docs.google.com/document/d/1FHjOQHUzUGmqe20xRl5IDQNjBWumTP7T/edit?usp=drive_link&oid=108394895055428723586&rtpof=true&sd=true

Before calling it **v1.0 Baseline** , we should close these items:

- Add the proposed requirements FR-039–FR-043, BR-021, and NFR-012–NFR-016 to the SRS.
- Update the requirements-to-features traceability matrix accordingly.
- Replace [Student Name] and add approval/reviewer information.
- Confirm production-specific details that are currently represented by architectural assumptions:
 - Exact K-Bank ECR and PSP protocols, Receipt-printer model and communication protocol, RabbitMQ hosting provider
 - Google Cloud region, Inventory message schema and acknowledgement rules
- Obtain stakeholder approval for the architectural decisions and risks.

The following are intentionally not included because they belong in **later feature-level design** specifications:

- Complete REST endpoint specifications
- Detailed database schema and Prisma models
- Screen designs and interaction flows
- Exact receipt layout
- Class/component designs
- Detailed test scenarios
- Deployment scripts and infrastructure code



TikTockIT-System-Level-SDS-Draft.docx



TokTickIT-System-Level-SDS-v1.0.docx

6. Per Feature: Engineering Contract

SDS + UI/API/Data Specs + **Acceptance Criteria** + STS

For Each Feature

- UI design specs
- API design specs
- Data model
- API Tests
- Unit Tests to cover all Functional Requirements and Business Rules
- UI Tests

7. Per Feature: AI Agent Implementation

1. Focus on individual components
2. Goal: a working, flexible, robust, ... piece of software

Recommended Design Process

1. **SRS.** FR, BR, NFR.



2. **Complete Feature Inventory.** Feature-A, Feature-B, Feature-C, ...



3. **Requirements-to-Features Traceability.** Map every FR, BR, and NFR to features. FR-003 → Feature-E.



4. **System-Level SDS.** Architecture, tech stack, UI standards, API conventions, security, deployment



5. **Select the Sprint Vertical Slice.** Choose a “Feature Bundle” to be implemented in the sprint



6. **Feature-Level Engineering Contract:** SDS + UI/API/Data Specs + Acceptance Criteria + STS



7. **AI Agent Implementation.** Implement one bounded Issue at a time according to the specifications



8. **Test and Human Review.** Run automated tests, inspect the code and Git diff, correct defects.



9. **Pull Request and Integration.** Review, approve, and merge feature branch into the Sprint staging branch



10. **Sprint Integration, System Testing, and UAT.** Run tests on Sprint staging branch. If PASS: Pull Request from Sprint staging → main. If FAIL: create a fix branch on Sprint staging branch, update the SDS/STS if needed, perform Steps 7-9, and repeat Step 10.



11. **Per Release: Deploy main Branch to Production**

per feature

8. Tests and Human Review. Example: Order Form in E-Commerce System

Unit → Integration → System → E2E → UAT

Test	Concrete Scope in This Sprint	Example	Goal
1. Unit Test	Test one small function, class, component, service, or module in isolation. External dependencies are mocked/stubbed.	Test <code>calculateOrderTotal()</code> , discount calculation, shipping fee calculation, field validation, or a React OrderForm component's validation logic.	Verify each unit works in isolation. Does this specific feature satisfy its requirements?
2. Integration Test	Test whether two or more implemented parts work correctly together. Usually includes real interfaces between modules, DB, or APIs.	Submit order data through the Order Service and verify that the correct Order and OrderItem records are written to the database. Or test Order Service + Inventory API integration.	Do the components and features work together?
3. System Test	Test the completed sprint increment as part of the whole running application against its requirements.	Start the customer app, create an order, validate required fields, calculate totals, save the order, and verify the correct confirmation is shown. Also test errors: invalid quantities, unavailable products, ...	Check functional and non-functional requirements (performance, security, etc.)
4. E2E Test	Test a realistic user workflow through all relevant layers, normally from UI to backend to DB/external services.	Customer browses products → adds products to cart → opens Order Form → enters shipping information → confirms order → system saves order → confirmation page shows order number.	Does the integrated system satisfy meet requirements as far as IT team is concerned.
5. UAT	A Product Owner/customer/business representative checks whether the delivered increment actually satisfies business need and acceptance criteria.	Product Owner tries the Order Form and confirms: "Yes, this is how customers should place an order, the required information is captured, totals are correct, and the flow is acceptable."	Does the integrated system satisfy the stakeholder's business expectations? Get users to sign off.

Per Feature vs. Per Sprint

Document or activity	Scope
Feature-Level SDS	Per feature
Feature STS	Per feature
Feature test execution	Per Issue or feature branch
Sprint integration test specification	Per Sprint
Sprint integration, regression, and UAT	Per Sprint
Production smoke test	Per release

THE END

つづく

tsuzuku